Developer Documentation
Campus Marketplace

Proposed by

Group 3

Group Leaders

Daniël de Jager – 41669436

Pieter Meyer – 42937582

Group Members

Petrus Daniel Bekker – 37267027

Johann Nell – 38609029

Thabiso Nkashe – 38614898

Dimpho Malatji – 42705576

Bradley Motsiri – 29998360

Delius Schoeman – 45616779

Tiyane Sibuya – 42591570

Reese Van Der Linde – 46859675

Meiring van Niekerk – 47817909

Daniel Van Wyk – 33505136

Sithole Zakhele - 36081868

17 October 2025

Module: CMPG 323

Project Manager: Ruan van Heerden – 41763882

Portfolio Manager: Henri van Rensburg

Table of Contents

1 Project Overview 4
1.1 Core Features 4
1.2 Technology Stack 4
2 Getting Started Guide 4
2.1 Quick Setup 4
2.1.1 Clone and install dependencies 4
2.1.2 Copy environment template and configure with your credentials 4
2.1.3 Start development server with Turbopack for faster builds 5
2.2 Environment Configuration 5
2.2.1 Firebase Configuration 5
2.2.2 Firebase Admin SDK (Server-side only) 5
2.2.3 PayFast Payment Gateway 6
2.2.4 Admin Configuration 6
2.3 Required NPM Dependencies 6
2.3.1 Core Framework & React 6
2.3.2 Firebase & Authentication 6
2.3.3 PayFast Payment Integration 6
2.3.4 UI Components & Styling 6
2.3.5 Charts & Analytics 7
2.3.6 Form Handling & Validation 8
2.3.7 Notifications & UI Enhancements 8
2.3.8 Development Dependencies 8
3 Authentication System (/src/app/login/page.tsx) 8
3.1 Login Component Implementation 8
4 Authentication Context (/src/context/auth.tsx) 11
4.1 Authentication Context Provider 11
4.2 Token Management System (/src/context/actions.ts) 14
4.2.1 Server Actions for Token Handling 14
4.3 Route Protection Middleware (/src/middleware.ts) 16
4.3.1 Authentication Middleware 16
4.4 Project Configuration & Scripts 18
4.4.1 Package.json Scripts 18
4.4.2 Next.js Configuration (next.config.ts) 18
4.4.3 TypeScript Configuration (tsconfig.json) 19
5 API Routes (/src/app/api/items/route.ts) 21
5.1 Items API - Fetch with Filtering (/src/app/api/items/list/route.ts) 21
5.2 Create Item API - Add new item (/src/app/api/items/create/route.ts) 23
6 PayFast Integration (/src/app/api/payfast/checkout/route.ts) 26
6.1 PayFast Checkout API 26
7 Database Schema (/src/types/item.ts) 28
7.1 Firestore Collections Structure 28

8 Core Components (/src/components/ItemCard.tsx) 29
8.1 Item Display Component (from SearchClient) 29
9 Search Functionality (/src/app/search-client.tsx) 31
9.1 Search Implementation with Filtering 31
10 PayFast Security Implementation (/src/app/api/payfast/checkout/route.ts) 33
10.1 PayFast Security: Signature Generation & Validation 33
10.1.1 Signature Generation in Checkout Route (/src/app/api/payfast/checkout/route.ts) 33
10.1.2 Signature Validation in Notify Route (/src/app/api/payfast/notify/route.ts) 34
11 Form Validation (/src/validation/itemSchema.ts) 35
11.1 Item Validation Schema with Zod 35
12 Project Structure 36
13 Admin Dashboard & Analytics (/src/app/admin/page.tsx) 39
13.1 Real Admin Dashboard Implementation 39
14 Shopping Cart System (/src/app/cart/page.tsx) 42
14.1 Real Cart Implementation with VAT 42
15 Image Upload System (/src/components/multi-image-uploader.tsx) 47
15.1 Real Multi-Image Uploader Implementation 47
16 Additional Components 50
16.1 Add to Cart Button (/src/components/AddToCartButton.tsx) 50
16.2 Authentication Buttons (/src/components/auth-buttons.tsx) 52
17 Custom Hooks 54
17.1 Session Timeout Hook (/src/hooks/useSessionTimeout.ts) 54
18 API Routes - Complete Implementation 57
18.1 Admin Status API (/src/app/api/admin/status/route.ts) 57
18.2 Cart Management API (/src/app/api/cart/route.ts) 58
19 Utility Functions 60
19.1 Image URL Formatter (/src/lib/imageUrlFormatter.ts) 60
20 Firebase Storage Security Rules (/storage.rules) 61
20.1 Production Storage Rules Implementation 61
21 Environment Variables Configuration (/.env.example) 63
21.1 Complete Environment Template 63
22 Deployment Instructions (/firebase.json) 64
22.1 Firebase Hosting Configuration 64
22.2 Production Deployment Steps 65
23 API Error Handling Patterns 66
23.1 Standardized Error Response Format 66
23.2 Error Handling Implementation Examples 67

1 Project Overview

Campus Marketplace is a comprehensive e-commerce platform designed specifically for university students to buy and sell items within their campus community.

1.1 Core Features

Secure Authentication: Firebase Auth with email/password and Google OAuth integration

Payment Processing: PayFast integration with signature verification

Shopping Cart: Full cart management with VAT calculations

Admin Dashboard: Analytics and user moderation

Image Management: Firebase Storage integration

1.2 Technology Stack

Frontend: Next.js 15.4.6 with React 19.1.0 and TypeScript

Backend: Next.js API routes with Firebase Admin SDK

Database: Cloud Firestore

Authentication: Firebase Auth

Payments: PayFast with crypto-js

Styling: Tailwind CSS with Radix UI

2 Getting Started Guide

2.1 Quick Setup

```bash

### *2.1.1 Clone and install dependencies*

git clone https://github.com/Pieter246/campus-marketplace.git

cd campus-marketplace

npm install

### *2.1.2 Copy environment template and configure with your credentials*

cp .env.example .env.local
```

2.1.3 Start development server with Turbopack for faster builds

npm run dev

Explanation:

This streamlined setup process ensures developers can quickly establish a local development environment. The project leverages Next.js 15.4.6's enhanced development server with Turbopack, which provides significantly faster hot reloading and builds times compared to traditional webpack builds. The environment configuration step is critical as it establishes connections to Firebase services (authentication, database, storage) and Pay Fast payment gateway. The `.env.example` file serves as a template containing all required environment variables with placeholder values, ensuring developers don't miss any critical configuration steps. This approach follows modern development best practices by separating configuration from code and enabling easy deployment across different environments (development, staging, production).

Physical Code Explanation:

Line 2: `git clone <repository-url>` clones the Git repository to local machine, replacing `<repository-url>` with actual repository URL. Line 3: `cd campus-marketplace` changes directory into the project folder. Line 4: `npm install` installs all dependencies listed in `package.json` into `node_modules` folder. Line 7: `cp .env.example .env.local` copies environment template file to local configuration file (`.env.local` is ignored by Git for security). Line 10: `npm run dev` executes the development script from `package.json`, starting Next.js dev server with Turbopack bundler for faster builds.

2.2 Environment Configuration

```env.local
```

2.2.1 Firebase Configuration

NEXT_PUBLIC_FIREBASE_API_KEY=your_api_key

NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your_project.firebaseapp.com

NEXT_PUBLIC_FIREBASE_PROJECT_ID=your_project_id

NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=your_project.appspot.com

NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your_sender_id

NEXT_PUBLIC_FIREBASE_APP_ID=your_app_id

2.2.2 Firebase Admin SDK (Server-side only)

FIREBASE_PRIVATE_KEY=your_private_key

FIREBASE_CLIENT_EMAIL=your_client_email

2.2.3 PayFast Payment Gateway

PAYFAST_MERCHANT_ID=your_merchant_id

PAYFAST_MERCHANT_KEY=your_merchant_key

PAYFAST_PASSPHRASE=your_passphrase

NEXT_PUBLIC_BASE_URL=http://localhost:3000

2.2.4 Admin Configuration

ADMIN_EMAILS=admin1@example.com,admin2@example.com

```

## ***2.3 Required NPM Dependencies***

```bash

### *2.3.1 Core Framework & React*

npm install next@15.4.6 react@19.1.0 react-dom@19.1.0

npm install typescript@5.9.2 @types/react@19 @types/react-dom@19

### *2.3.2 Firebase & Authentication*

npm install firebase@12.1.0 firebase-admin@13.5.0

npm install jose@6.1.0

### *2.3.3 PayFast Payment Integration*

npm install crypto-js@4.2.0 @types/crypto-js@4.2.2

npm install qs@6.14.0 @types/qs@6.9.11

### *2.3.4 UI Components & Styling*

npm install @radix-ui/react-alert-dialog@1.1.15

npm install @radix-ui/react-avatar@1.1.10

npm install @radix-ui/react-dropdown-menu@2.1.16

npm install @radix-ui/react-label@2.1.7

npm install @radix-ui/react-select@2.2.6

npm install @radix-ui/react-slot@1.2.3

npm install @radix-ui/react-tabs@1.1.13

npm install tailwindcss@4 @tailwindcss/postcss@4.1.12

npm install tailwind-merge@3.3.1 clsx@2.1.1

npm install lucide-react@0.544.0

### *2.3.5 Charts & Analytics*

npm install recharts@3.2.1

npm install numeral@2.0.6 @types/numeral@2.0.5

**Explanation**:

This dependency configuration represents a production-ready technology stack optimized for modern marketplace development. The Next.js 15.4.6 framework provides the foundation with App Router, server components, and enhanced performance features. React 19.1.0 introduces concurrent features and improved hydration for better user experience. The Firebase ecosystem enables full-stack functionality: client SDK for authentication and real-time data, Admin SDK for server-side operations, and jose for secure JWT handling. PayFast integration requires crypto-js for signature generation (essential for South African payment processing) and as for proper query string formatting. Radix UI components provide accessible, headless UI primitives that integrate seamlessly with Tailwind CSS for consistent design systems. The Tailwind v4 setup includes PostCSS integration and utility libraries for dynamic class management. Recharts delivers interactive data visualization for the admin dashboard, while numeral provides localized number formatting for South African Rand currency display. This stack ensures type safety, performance, accessibility, and maintainability for enterprise-grade marketplace applications.

**Physical Code Explanation**:

Lines 2-3: `npm install` commands with package names and specific version numbers using `@` syntax - `next@15.4.6` installs Next.js framework version 15.4.6, `react@19.1.0` and `react-dom@19.1.0` install React library and DOM bindings, `typescript@5.9.2` adds TypeScript compiler, `@types/react@19` and `@types/react-dom@19` provide TypeScript type definitions. Lines 5-7: Firebase packages - `firebase@12.1.0` for client SDK, `firebase-admin@13.5.0` for server-side operations, `jose@6.1.0` for JWT token handling. Lines 9-11: PayFast integration - `crypto-js@4.2.0` for MD5 hashing with TypeScript types, `qs@6.14.0` for query string parsing with types. Lines 13-18: Radix UI components with version pinning using `@` notation for consistent styling and accessibility. Lines 19-21: Tailwind CSS v4 with PostCSS plugin, utility libraries `tailwind-merge` for conditional classes and `clsx` for class concatenation, `lucide-react` for iconography. Lines 23-24: Data visualization with `recharts@3.2.1` for charts and `numeral@2.0.6` for number formatting with TypeScript definitions.

### *2.3.6 Form Handling & Validation*

npm install react-hook-form@7.62.0

npm install @hookform/resolvers@5.2.1

npm install zod@4.1.8

### *2.3.7 Notifications & UI Enhancements*

npm install sonner@2.0.7

npm install react-hot-toast@2.6.0

npm install @hello-pangea/dnd@18.0.1

npm install embla-carousel-react@8.6.0

### *2.3.8 Development Dependencies*

npm install --save-dev eslint@9 eslint-config-next@15.4.6

npm install --save-dev @types/node@20.19.11

```

Explanation:

The environment configuration represents the cornerstone of the application's external service integration strategy. Firebase configuration variables prefixed with `NEXT_PUBLIC_` are intentionally exposed to the client-side code, enabling direct browser-to-Firebase communication for real-time features like authentication state changes and live data updates. The Firebase project ID connects the application to a specific Firestore database instance, while the auth domain handles OAuth redirects and email verification links. PayFast credentials are server-side only for security, enabling the application to process South African Rand payments securely. The merchant passphrase acts as a cryptographic key for generating payment signatures, preventing transaction tampering. This configuration strategy ensures secure service integration while maintaining the flexibility to deploy across multiple environments with different service endpoints and credentials.

3 Authentication System (/src/app/login/page.tsx)

3.1 Login Component Implementation

```typescript
export default function LoginForm() {
```
```

```
  const router = useRouter();
  const auth = useAuth();

  const [form, setForm] = useState({ email: "", password: "" });
  const [errors, setErrors] = useState<{ [key: string]: string }>({});
  const [loading, setLoading] = useState(false);

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setForm({ ...form, [e.target.name]: e.target.value });
    setErrors((prev) => ({ ...prev, [e.target.name]: "" }));
  };

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();

    const newErrors: { [key: string]: string } = {};
    if (!form.email.includes("@")) newErrors.email = "Please enter a valid
email address";
    if (!form.password) newErrors.password = "Password is required";

    setErrors(newErrors);
    if (Object.keys(newErrors).length > 0) return;

    setLoading(true);

    try {
      const { signInWithEmailAndPassword } = await import("firebase/auth");
      const { auth: clientAuth } = await import("@/firebase/client");
      const userCredential = await signInWithEmailAndPassword(clientAuth,
form.email, form.password);
      const user = userCredential.user;
      await user.reload();
      if (!user.emailVerified) {
        await clientAuth.signOut();
        toast.error("Error!", {
          description: "Your email is not verified. Please check your inbox
and click the verification link. If you did not receive it, you can resend
below."
        });
        router.push(`/auth/verify-
email?email=${encodeURIComponent(form.email)}`);
        return;
      }
      router.push("/");
    } catch (e: unknown) {
      let errorMessage = "An error occurred";
      if (typeof e === "object" && e !== null && "message" in e && typeof
e.message === "string") {
        errorMessage = e.message;
      }
      toast.error("Error!", {
        description: errorMessage,
```

```
      });
    } finally {
      setLoading(false);
    }
  };

  // Google OAuth Login
  const handleGoogleLogin = async () => {
    setLoading(true);
    try {
      await auth.loginWithGoogle();
      toast.success("Success!", { description: "Logged in successfully" });
      router.push("/");
    } catch (error: unknown) {
      const errorMessage: string =
        typeof error === "object" && error !== null && "code" in error &&
error.code === "auth/popup-closed-by-user"
          ? "You closed the popup before completing login."
          : typeof error === "object" && error !== null && "message" in error
&& typeof error.message === "string"
          ? error.message
          : "Google login failed";
      toast.error("Google login failed", { description: errorMessage });
    } finally {
      setLoading(false);
    }
  };
}
```

Explanation:

This authentication implementation demonstrates comprehensive form validation, error handling, and state management patterns essential for production applications. The form validation checks for email format and required fields before making API calls, reducing unnecessary network requests and providing immediate user feedback. The dynamic import strategy for Firebase modules enables code splitting, reducing the initial JavaScript bundle size - authentication code only loads when users actually attempt to log in. The email verification enforcement adds critical security by preventing unverified accounts from accessing the platform, while the toast notifications provide clear user feedback using the Sonner library. The Google OAuth implementation with `auth.loginWithGoogle()` handles popup authentication flows and automatic Firestore user document creation. Error handling distinguishes between different failure scenarios (user closing popup, invalid credentials, network issues) and provides appropriate user messaging. The loading states prevent double submissions and provide visual feedback during authentication processes. The component integrates seamlessly with the auth context and Next.js routing for smooth user experience transitions.

Physical Code Explanation:

Line 1: `"use client"` directive tells Next.js this component runs in the browser, not server-side. Lines 3-10: Import statements bring in React hooks (`useState` for component state),

Next.js navigation (`useRouter` for programmatic routing, `Link` for navigation), custom components (`Button`, `Input`, `Logo`), authentication context (`useAuth` for user state), and toast notifications (`toast` from sonner). Line 12: `export default function LoginForm()` declares the main component function. Line 13: `const router = useRouter()` creates a router instance for navigation. Line 14: `const auth = useAuth()` accesses the authentication context. Line 16: `const [form, setForm] = useState({ email: "", password: "" })` initializes form state with empty email and password strings. Line 17: `const [errors, setErrors] = useState<{ [key: string]: string }>({})` creates error state as an object with string keys and values. Line 18: `const [loading, setLoading] = useState(false)` manages loading state during form submission. Lines 20-23: `handleChange` function updates form state when input values change - `setForm({ ...form, [e.target.name]: e.target.value })` spreads existing form data and updates the specific field using bracket notation, while `setErrors((prev) => ({ ...prev, [e.target.name]: "" }))` clears any existing error for that field. Lines 25-34: `handleLogin` function handles form submission - `e.preventDefault()` stops default form submission, then validates email contains "@" symbol and password exists, storing errors in `newErrors` object. Lines 35-36: `setErrors(newErrors)` updates error state and early return if validation fails. Lines 38-49: Async Firebase authentication - `setLoading(true)` shows loading state, dynamic imports load Firebase modules only when needed, `signInWithEmailAndPassword()` attempts authentication with form data, `user.reload()` refreshes user data, and email verification check redirects unverified users to verification page.

4 Authentication Context (/src/context/auth.tsx)

4.1 Authentication Context Provider

```typescript
export const AuthProvider = ({ children }: { children: React.ReactNode }) => {
  const [currentUser, setCurrentUser] = useState<User | null>(null)
  const [customClaims, setCustomClaims] = useState<CustomClaims | null>(null)
  const [loading, setLoading] = useState(true)
  const [isAdmin, setIsAdmin] = useState(false)

  // Function to check admin status via API
  const refreshAdminStatus = useCallback(async () => {
    try {
      if (!currentUser) {
        setIsAdmin(false)
        return
      }

      const token = await currentUser.getIdToken()
      const response = await fetch('/api/admin/status', {
        headers: {
          'Authorization': `Bearer ${token}`
        }
      })

      if (response.ok) {
        const data = await response.json()
```

```
        setIsAdmin(data.isAdmin || false)
      } else {
        setIsAdmin(false)
      }
    } catch (error) {
      console.error('Error checking admin status:', error)
      setIsAdmin(false)
    }
  }, [currentUser])

  useEffect(() => {
    const unsubscribe = auth.onAuthStateChanged(async (user) => {
      setCurrentUser(user ?? null)

      if (user) {
        const tokenResult = await user.getIdTokenResult()
        const token = tokenResult.token
        const refreshToken = user.refreshToken
        const claims = tokenResult.claims as CustomClaims

        setCustomClaims(claims ?? null)

        if (token && refreshToken) {
          await setToken({ token, refreshToken })
        }
      } else {
        await removeToken()
        setIsAdmin(false)
      }

      setLoading(false)
    })

    return () => unsubscribe()
  }, [])

  const loginWithGoogle = async (): Promise<User> => {
    const provider = new GoogleAuthProvider()
    const result = await signInWithPopup(auth, provider)
    const user = result.user

    if (!user) throw new Error("No user returned from Google login")

    // Call server action to create Firestore document if missing
    await registerGoogleUser({
      uid: user.uid,
      email: user.email!,
      emailVerified: user.emailVerified,
      displayName: user.displayName || "",
    })

    // Refresh admin status after registration
```

```
    setTimeout(() => refreshAdminStatus(), 1000)

    return user
  }
}

  const login = async (email: string, password: string) => {
    return signInWithEmailAndPassword(auth, email, password);
  };

  return (
    <AuthContext.Provider value={{ currentUser, login, loading }}>
      {!loading && children}
    </AuthContext.Provider>
  );
}
```

Explanation:

The AuthProvider represents a sophisticated state management solution that handles the complexities of Firebase authentication across the entire application. The `refreshAdminStatus` function demonstrates real-time role checking via API calls, ensuring admin privileges are verified server-side rather than relying on client-side claims. The `onAuthStateChanged` listener provides Firebase's recommended approach for detecting authentication state changes, automatically handling token refresh, session expiration, and cross-tab synchronization. The token and refresh token management via server actions ensures secure storage in HTTP-only cookies, preventing XSS attacks while maintaining session persistence. The Google OAuth integration with `registerGoogleUser` server action creates Firestore user documents automatically, ensuring consistent user data across authentication methods. The loading state prevents the "flash of unauthenticated content" problem during initial auth checks. This architecture supports complex authentication flows including role-based access control, automatic redirects, and seamless integration with protected API endpoints.

Physical Code Explanation:

Line 1: `"use client"` enables client-side React features. Lines 3-11: Import Firebase auth functions (`GoogleAuthProvider`, `signInWithEmailAndPassword`, etc.), React hooks (`createContext`, `useContext`, `useEffect`, `useState`, `useCallback`), and server actions (`removeToken`, `setToken`, `registerGoogleUser`). Lines 14-16: TypeScript type definition for custom claims extends `ParsedToken` with optional `admin` boolean property. Lines 18-27: `AuthContextType` interface defines the shape of authentication context including user state, functions, and admin status. Line 29: `createContext<AuthContextType | null>(null)` creates React context with null default. Line 31: `AuthProvider` component function accepts children prop. Lines 32-35: State variables using `useState` - `currentUser` holds Firebase user object, `customClaims` stores JWT claims, `loading` tracks authentication status, `isAdmin` manages admin privileges. Lines 38-58: `refreshAdminStatus` function uses `useCallback` for performance optimization, checks if user exists, gets ID token with `await currentUser.getIdToken()`, makes API call to `/api/admin/status` with Bearer token in Authorization header, parses response and updates `isAdmin` state. Lines 60-84: `useEffect` hook subscribes to Firebase auth changes with `auth.onAuthStateChanged()`,

extracts user, token, and refresh token, calls `setToken` server action to store in HTTP-only cookies, and calls `removeToken` on logout. Lines 87-92: Second `useEffect` watches `currentUser` changes, calls `refreshAdminStatus()` when user logs in, sets up 60-second interval for periodic admin status checks.

4.2 Token Management System (/src/context/actions.ts)

4.2.1 Server Actions for Token Handling

```typescript
import { cookies } from "next/headers";
import { NextRequest, NextResponse } from "next/server";
import { decodeJwt } from "jose";

export async function middleware(request: NextRequest) {
    console.log("MIDDLEWARE: ", request.url);
    if(request.method === "POST"){
        return NextResponse.next();
    }

    const cookieStore = await cookies();
    const token = cookieStore.get("firebaseAuthToken")?.value;
    const hasSession = cookieStore.get("authSession")?.value;

    const { pathname } = request.nextUrl;

    // Allow unauthenticated access to public routes
    if(!token &&
        (pathname.startsWith("/login") ||
        pathname.startsWith("/register") ||
        pathname.startsWith("/reset-password") ||
        pathname.startsWith("/item-search") ||
        pathname.startsWith("/item"))
    ){
        return NextResponse.next();
    }

    // Redirect unauthenticated users to login
    if (!token) {
        return NextResponse.redirect(new URL("/login", request.url));
    }

    try {
        const decodedToken = decodeJwt(token);
        const currentTime = Math.floor(Date.now() / 1000);

        // Check if token is expired
        if (decodedToken.exp && decodedToken.exp < currentTime) {
            return NextResponse.redirect(new URL("/login", request.url));
        }
```

```
        // Check admin routes
        if (pathname.startsWith("/admin") && !decodedToken.admin) {
            return NextResponse.redirect(new URL("/", request.url));
        }

        return NextResponse.next();
    } catch (error) {
        console.error("Token validation error:", error);
        return NextResponse.redirect(new URL("/login", request.url));
    }
}

export const config = {
    matcher: [
        "/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt)
.*)",
    ],
};
```

Explanation:

This token management system implements secure server-side authentication handling using Next.js server actions and HTTP-only cookies. The `setToken` function performs critical security operations: it verifies the Firebase ID token server-side to prevent token forgery, automatically assigns admin claims based on email configuration, and stores tokens in secure HTTP-only cookies that cannot be accessed by client-side JavaScript. This approach prevents XSS attacks while maintaining session persistence across browser refreshes. The admin role assignment logic checks against environment-configured admin emails and uses Firebase's custom claims system for role-based access control. The cookie configuration uses the `secure` flag in production to ensure tokens are only transmitted over HTTPS connections. The `removeToken` function provides secure logout functionality by clearing both authentication and refresh tokens. This implementation supports automatic token refresh, session management, and secure role-based authorization throughout the application.

Physical Code Explanation:

Line 1: `"use server"` directive enables Next.js server actions for server-side execution. Lines 3-4: Import Firebase Admin SDK auth instance and Next.js cookies helper. Lines 6-10: `removeToken` async function - awaits `cookies()` to get cookie store, calls `delete()` method to remove both "firebaseAuthToken" and "firebaseAuthRefreshToken" cookies. Lines 12-19: `setToken` function signature with destructured parameters for `token` and `refreshToken` strings. Lines 21-24: Token verification with `auth.verifyIdToken(token)` - validates JWT signature and expiration, returns early if verification fails. Lines 26-27: Admin email configuration - splits `ADMIN_EMAILS` environment variable by commas using `split(",")`, defaults to empty array with nullish coalescing `?? []`. Lines 28-36: Admin role assignment logic - gets user record with `auth.getUser()`, checks if user email exists in admin list using `includes()`, verifies user doesn't already have admin claim, calls `auth.setCustomUserClaims()` to assign admin role. Lines 38-47: Cookie storage - gets cookie store with `await cookies()`, sets "firebaseAuthToken" and

"firebaseAuthRefreshToken" with `httpOnly: true` for security, `secure` flag based on production environment check.

4.3 Route Protection Middleware (/src/middleware.ts)

4.3.1 Authentication Middleware

```typescript
import { cookies } from "next/headers";
import { NextRequest, NextResponse } from "next/server";
import { decodeJwt } from "jose";

export async function middleware(request: NextRequest) {
    console.log("MIDDLEWARE: ", request.url);
    if(request.method === "POST"){
        return NextResponse.next();
    }

    const cookieStore = await cookies();
    const token = cookieStore.get("firebaseAuthToken")?.value;
    const hasSession = cookieStore.get("authSession")?.value;

    const { pathname } = request.nextUrl;

    // Allow unauthenticated access to public routes
    if(!token &&
        (pathname.startsWith("/login") ||
        pathname.startsWith("/register") ||
        pathname.startsWith("/reset-password") ||
        pathname.startsWith("/item-search") ||
        pathname.startsWith("/item"))
    ){
        return NextResponse.next();
    }

    // Redirect unauthenticated users to login
    if (!token) {
        return NextResponse.redirect(new URL("/login", request.url));
    }

    try {
        const decodedToken = decodeJwt(token);
        const currentTime = Math.floor(Date.now() / 1000);

        // Check if token is expired
        if (decodedToken.exp && decodedToken.exp < currentTime) {
            return NextResponse.redirect(new URL("/login", request.url));
        }

        // Check admin routes
        if (pathname.startsWith("/admin") && !decodedToken.admin) {
```

```
            return NextResponse.redirect(new URL("/", request.url));
        }

        return NextResponse.next();
    } catch (error) {
        console.error("Token validation error:", error);
        return NextResponse.redirect(new URL("/login", request.url));
    }
}

export const config = {
    matcher: [
        "/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt)
.*)",
    ],
};
```

Explanation:

This middleware implements comprehensive route protection and authentication verification for the entire application. It runs before every request to protected routes, providing a security layer that validates user authentication and authorization. The public routes logic allows unauthenticated access to login, registration, and item browsing pages while protecting sensitive areas like user profiles and admin dashboards. The JWT token validation uses the `jose` library to decode and verify Firebase ID tokens, checking expiration times and custom claims for role-based access control. The admin route protection specifically validates the `admin` claim before allowing access to administrative functions. The middleware matcher configuration excludes API routes, static assets, and system files from authentication checks, ensuring optimal performance. This implementation supports automatic token refresh, prevents unauthorized access to protected resources, and provides seamless redirection flows for better user experience.

Physical Code Explanation:

Lines 1-3: Import Next.js `cookies` helper, request/response objects, and `decodeJwt` from jose library. Lines 5-9: Multi-line comment explaining middleware purpose and functionality. Lines 11-15: `middleware` function declaration with NextRequest parameter, console log for debugging, early return for POST requests with `NextResponse.next()`. Lines 17-19: Cookie extraction - await `cookies()` to get cookie store, extract "firebaseAuthToken" value with optional chaining `?.value`, get "authSession" cookie for session tracking. Line 21: Destructure `pathname` from request URL using `request.nextUrl`. Lines 23-30: Public routes logic - check if no token exists AND pathname starts with allowed public routes using `pathname.startsWith()` for login, register, reset-password, item-search, and item pages, return `NextResponse.next()` to allow access. Lines 32-38: Authenticated user redirection - if token exists AND user tries to access auth pages (login/register/reset-password), redirect to home page using `NextResponse.redirect(new URL("/", request.url))`. Lines 40-45: Session race condition prevention - if session has been checked (`hasSession` is true) but no token exists, safely redirect to login page. Lines 47-48: Main route protection - if no token exists for protected routes, redirect to login (this is the primary security barrier).

4.4 Project Configuration & Scripts

4.4.1 Package.json Scripts

```json
{
  "scripts": {
    "dev": "next dev --turbopack",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "firebase:deploy": "firebase deploy",
    "firebase:deploy-rules": "firebase deploy --only
firestore:rules,storage:rules",
    "firebase:emulator": "firebase emulators:start --only firestore,storage",
    "firebase:test-rules": "firebase emulators:exec --only firestore 'npm run
test:rules'",
    "test:rules": "echo 'Add Firebase rules tests here'"
  }
}
```

Physical Code Explanation:

Lines 2-11: `"scripts"` object defines npm run commands - `"dev": "next dev --turbopack"` starts development server with Turbopack bundler for faster builds, `"build": "next build"` compiles production build, `"start": "next start"` runs production server, `"lint": "next lint"` runs ESLint code analysis, `"firebase:deploy": "firebase deploy"` deploys entire project to Firebase, `"firebase:deploy-rules": "firebase deploy --only firestore:rules,storage:rules"` deploys only security rules using `--only` flag with comma-separated services, `"firebase:emulator": "firebase emulators:start --only firestore,storage"` runs local Firebase emulators for development, `"firebase:test-rules": "firebase emulators:exec --only firestore 'npm run test:rules'"` executes rules tests in emulator environment, `"test:rules": "echo 'Add Firebase rules tests here'"` placeholder for future Firebase rules testing.

4.4.2 Next.js Configuration (next.config.ts)

```typescript
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  images: {
    domains: ['firebasestorage.googleapis.com'],
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'firebasestorage.googleapis.com',
        port: '',
        pathname: '/v0/b/**',
      },
```

```
    ],
  },
  experimental: {
    serverActions: {
      allowedOrigins: ['localhost:3000'],
    },
  },
  typescript: {
    ignoreBuildErrors: false,
  },
  eslint: {
    ignoreDuringBuilds: false,
  },
};

export default nextConfig;
```

Physical Code Explanation:

Line 1: `import type { NextConfig } from "next"` imports TypeScript type definition for Next.js configuration object. Line 3: `const nextConfig: NextConfig = {` declares configuration object with explicit typing. Lines 4-13: `images` configuration for optimization - `domains: ['firebasestorage.googleapis.com']` allows Firebase Storage images, `remotePatterns` array provides fine-grained control with `protocol: 'https'` enforcing secure connections, `hostname: 'firebasestorage.googleapis.com'` specifying exact domain, `port: ''` allowing default HTTPS port, `pathname: '/v0/b/**'` matching Firebase Storage path pattern with wildcard `**`. Lines 14-18: `experimental` features - `serverActions` object with `allowedOrigins: ['localhost:3000']` restricting server action origins for security. Lines 19-25: Build validation settings - `typescript: { ignoreBuildErrors: false }` ensures TypeScript errors fail builds, `eslint: { ignoreDuringBuilds: false }` ensures ESLint errors fail builds. Line 28: `export default nextConfig` exports configuration object as default export.

4.4.3 TypeScript Configuration (tsconfig.json)

```json
{
  "compilerOptions": {
    "lib": ["dom", "dom.iterable", "es6"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [
```

```
      {
        "name": "next"
      }
    ],
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Physical Code Explanation:

Lines 2-23: `"compilerOptions"` object configures TypeScript compiler behavior - `"lib": ["dom", "dom.iterable", "es6"]` includes DOM APIs and ES6 features, `"allowJs": true` enables JavaScript file compilation, `"skipLibCheck": true` skips type checking in declaration files for faster builds, `"strict": true` enables all strict type checking options, `"noEmit": true` prevents TypeScript from generating output files (Next.js handles compilation), `"esModuleInterop": true` enables interoperability between CommonJS and ES modules, `"module": "esnext"` uses latest ECMAScript module system, `"moduleResolution": "bundler"` uses bundler-compatible module resolution for Next.js 13+, `"resolveJsonModule": true` allows importing JSON files as modules, `"isolatedModules": true` ensures each file can be safely transpiled without type information from other files, `"jsx": "preserve"` preserves JSX syntax for Next.js to handle, `"incremental": true` enables incremental compilation for faster subsequent builds. Lines 15-19: `"plugins"` array with `"name": "next"` enables Next.js TypeScript plugin for enhanced IDE support. Lines 20-24: Path mapping configuration - `"baseUrl": "."` sets project root as base directory, `"paths": { "@/*": ["./src/*"] }` creates `@/` alias pointing to `src/` directory for clean imports. Line 25: `"include"` array specifies files to compile including Next.js environment types, all TypeScript/TSX files, and generated types. Line 26: `"exclude": ["node_modules"]` excludes dependencies from compilation.

Explanation:

The project configuration demonstrates production-ready setup patterns for modern Next.js applications. The package.json scripts include Turbopack for faster development builds, Firebase deployment automation, and emulator testing capabilities. The Next.js configuration enables Firebase Storage image optimization through the `images.domains` setting, crucial for marketplace applications with user-uploaded photos. The `remotePatterns` configuration provides fine-grained control over external image sources, improving security by whitelisting specific Firebase Storage patterns. Server actions configuration ensures secure API endpoints with allowed origins restrictions. The TypeScript configuration uses strict mode for maximum type safety, includes path mapping for clean imports (`@/` prefix), and enables incremental compilation for faster builds. The module resolution strategy uses "bundler" mode for optimal compatibility with Next.js 15 and modern bundling tools. This configuration supports advanced features like image optimization, server-side rendering, and seamless

Firebase integration while maintaining development productivity and production performance.

5 API Routes (/src/app/api/items/route.ts)

5.1 Items API - Fetch with Filtering (/src/app/api/items/list/route.ts)

```typescript
export async function POST(req: NextRequest) {
  try {
    const body = await req.json();

    const {
      sellerId,
      buyerId,
      minPrice,
      maxPrice,
      condition,
      status,
      searchTerm,
      sort = "newest",
      category,
      page = 1,
      pageSize = 10,
    } = body;

    const user = await authenticateRequest(req);

    // Only enforce auth for private filters (sellerId or buyerId)
    if ((sellerId || buyerId) && !user) {
      return NextResponse.json({ message: "Unauthorized" }, { status: 401 });
    }

    let query: Query<DocumentData> = firestore.collection("items");

    // Apply filters
    if (sellerId) query = query.where("sellerId", "==", sellerId);
    if (buyerId) query = query.where("buyerId", "==", buyerId);
    if (minPrice !== null && minPrice !== undefined)
      query = query.where("price", ">=", minPrice);
    if (maxPrice !== null && maxPrice !== undefined)
      query = query.where("price", "<=", maxPrice);
    if (condition) {
      if (condition === "all") {
        query = query.where("condition", "in", ["new", "excellent", "used",
"fair", "poor"]);
      } else {
        query = query.where("condition", "==", condition);
      }
    }
    if (Array.isArray(status) && status.length > 0) {
```

```
      query = query.where("status", "in", status);
    }
    if (category && category !== "all") {
      query = query.where("category", "==", category);
    }

    // Apply sort logic
    if (sort === "price-asc") {
      query = query.orderBy("price", "asc");
    } else if (sort === "price-desc") {
      query = query.orderBy("price", "desc");
    } else if (sort === "oldest") {
      query = query.orderBy("updatedAt", "asc");
    } else {
      query = query.orderBy("updatedAt", "desc"); // default: newest
    }

    // Get total count for pagination
    const totalItems = (await query.count().get()).data().count;
    const totalPages = Math.ceil(totalItems / pageSize);

    // Apply pagination
    query = query.offset((page - 1) * pageSize).limit(pageSize);

    // Fetch items
    const snapshot = await query.get();

    let items = snapshot.docs.map((doc) => {
      const data = doc.data();
      return {
        id: doc.id,
        ...data,
        postedAt: data.postedAt?.toDate?.()?.toISOString(),
        updatedAt: data.updatedAt?.toDate?.()?.toISOString(),
      } as Item;
    });

    return NextResponse.json({ success: true, items, totalPages, currentPage:
page });
  } catch (error) {
    console.error('Items API error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500
});
  }
}
```

Explanation:

This API endpoint exemplifies comprehensive database querying patterns optimized for marketplace applications. The POST method allows complex filtering options sent in the request body, supporting advanced search functionality with multiple parameters. The

authentication enforcement is conditional - only private operations (viewing your own listings) require authentication, while public browsing remains accessible. The Firestore query building approach starts with a base collection and conditionally adds filters, ensuring only relevant items are retrieved from the database. This pattern leverages Firestore's indexed queries for optimal performance. The comprehensive filtering system supports price ranges, condition states, categories, and item status - enabling powerful search experiences. The sorting implementation covers common marketplace needs: newest, oldest, price ascending/descending. The pagination system calculates total pages and provides offset-based navigation, essential for performance with large datasets. The response format includes both items and pagination metadata, enabling frontend components to render page controls and total counts. Document timestamp conversion to ISO strings ensures consistent date handling across the application.

Physical Code Explanation:

Lines 1-4: Import statements for Next.js request/response objects, Firebase authentication helper, Item type, and Firestore Query types. Line 6: `export async function POST(req: NextRequest)` defines async POST handler. Line 8: `const body = await req.json()` parses request body JSON. Lines 10-18: Destructuring assignment extracts filter parameters from body object with default values (`sort = "newest"`, `page = 1`, `pageSize = 10`). Line 20: `const user = await authenticateRequest(req)` validates JWT token and returns user object or null. Lines 22-25: Conditional authentication check - if `sellerId` or `buyerId` filters requested but no authenticated user, return 401 Unauthorized. Line 27: `let query: Query<DocumentData> = firestore.collection("items")` initializes base Firestore query. Lines 29-47: Conditional query building - each `if` statement checks parameter existence and chains additional `where()` clauses: `query.where("sellerId", "==", sellerId)` filters by seller, `query.where("price", ">=", minPrice)` sets minimum price, `query.where("condition", "in", array)` handles multiple conditions, `query.where("status", "in", status)` filters by status array. Lines 42-46: Category filtering with special "all" case - if category is "all", no filter applied; otherwise `query.where("category", "==", category)` filters specific category.

5.2 Create Item API - Add new item (/src/app/api/items/create/route.ts)

```typescript
import { authenticateRequest, firestore } from "@/firebase/server";
import { Timestamp } from "firebase-admin/firestore";
import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";

// Zod schema for validation
const CreateItemSchema = z.object({
  title: z.string().min(1),
  collectionAddress: z.string().min(1),
  description: z.string().min(1),
  price: z.number().nonnegative(),
  status: z.enum(["draft", "pending", "for-sale", "sold", "withdrawn"]),
  condition: z.enum(["new", "excellent", "used", "fair", "poor"]),
```

```
  category: z.enum(["books", "electronics", "clothing", "notes", "stationery",
"other"]),
});

export async function POST(req: NextRequest) {
  try {
    const user = await authenticateRequest(req);
    if (!user) {
      return NextResponse.json({ message: "Unauthorized" }, { status: 401 });
    }

    const raw = await req.json();
    const parsed = CreateItemSchema.safeParse(raw.rest ?? raw);

    if (!parsed.success) {
      console.warn("Validation failed:", parsed.error.flatten());
      return NextResponse.json(
        {
          message: "Invalid input",
          issues: parsed.error.flatten().fieldErrors,
        },
        { status: 400 }
      );
    }

    // Create the item
    const item = await firestore.collection("items").add({
      ...parsed.data,
      sellerId: user.uid,
      postedAt: Timestamp.now(),
      updatedAt: Timestamp.now(),
      viewsCount: 0,
    });

    console.log(`Item created: ${item.id}`);

    // Display message item created and return the item Id
    return NextResponse.json(
      {
        success: true,
        message: "Item created successfully",
        item: { itemId: item.id },
```

```
      },
      { status: 201 }
    );
  } catch (error: unknown) {
    console.error("Item creation error:", error);
    const errorMessage: string = error instanceof Error ? error.message :
"Unknown error";
    return NextResponse.json(
      {
        message: "Internal server error",
        error: errorMessage,
      },
      { status: 500 }
    );
  }
}
```

Explanation:

This API endpoint creates new items in the marketplace with comprehensive validation and authentication. It validates user authentication, validates input data using Zod schema, creates the item document in Firestore, and returns the created item ID. The endpoint supports multiple item categories, conditions, and statuses while ensuring data integrity through strict validation.

Physical Code Explanation:

Lines 1-4: Import statements for Firebase server utilities (authentication and Firestore), Firebase Admin Timestamp class, Next.js request/response types, and Zod validation library. Lines 7-14: Zod schema definition with title (non-empty string), collectionAddress (pickup location string), description (item details string), price (non-negative number), status (enum for marketplace states), condition (enum for item quality), and category (enum for marketplace sections). Line 16: Async POST handler function export for item creation endpoint. Lines 17-21: Authentication check - calls `authenticateRequest(req)` to validate JWT token and returns 401 Unauthorized if user not authenticated. Lines 23-24: Request body parsing with `raw.rest` fallback and Zod schema validation using `safeParse()`. Lines 26-34: Validation error handling - logs validation failures and returns 400 Bad Request with detailed field error messages from Zod. Lines 37-42: Firestore document creation - spreads validated data and adds sellerId (user.uid), postedAt/updatedAt timestamps (Timestamp.now()), and viewsCount initialization (0). Lines 44-52: Success response with 201 Created status, success flag, confirmation message, and new item ID. Lines 53-62: Error handling - catches exceptions, logs errors, extracts error messages safely, and returns 500 Internal Server Error with error details.

6 PayFast Integration (/src/app/api/payfast/checkout/route.ts)

6.1 PayFast Checkout API

```typescript
export async function POST(req: NextRequest) {
  try {
    const { cart } = await req.json(); // get static cart from frontend
    if (!cart || cart.length === 0) {
      return NextResponse.json({ message: "Cart is empty" }, { status: 400 });
    }

    // Dummy user for sandbox
    const user = {
      displayName: "Test User",
      email: "student@example.com",
    };

    // Calculate total
    let totalAmount = 0;
    const itemNames: string[] = [];
    cart.forEach((item: { price: number; name: string }) => {
      // Treat each unique item as quantity = 1
      totalAmount += Number(item.price || 0);
      itemNames.push(item.name);
    });

    const paymentData: Record<string, string> = {
      merchant_id: MERCHANT_ID,
      merchant_key: MERCHANT_KEY,
      return_url: `${process.env.NEXT_PUBLIC_BASE_URL}/success`,
      cancel_url: `${process.env.NEXT_PUBLIC_BASE_URL}/cart`,
      notify_url: `${process.env.NEXT_PUBLIC_BASE_URL}/api/payfast/notify`,
      name_first: user.displayName,
      name_last: "",
      email_address: user.email,
      amount: totalAmount.toFixed(2),
      item_name: itemNames.join(", "),
    };

    // Create query string for signature generation
    const queryStringForSignature = Object.entries(paymentData)
      .filter(([, value]) => value !== null && value !== undefined && value
!== "")
      .map(([key, value]) =>
`${key}=${encodeURIComponent(String(value)).replace(/%20/g, "+")}`)
      .join("&");

    // Add passphrase
    const stringToHash =
`${queryStringForSignature}&passphrase=${PASSPHRASE}`;
```

```
    // Generate signature
    const signature = md5(stringToHash).toString();
    paymentData.signature = signature;

    // Create final query string for redirect URL
    const finalQueryString = qs.stringify(paymentData, { encode: true });

    const payfastUrl = SANDBOX
      ? `https://sandbox.payfast.co.za/eng/process?${finalQueryString}`
      : `https://www.payfast.co.za/eng/process?${finalQueryString}`;

    return NextResponse.json({ url: payfastUrl });
  } catch (err) {
    console.error("PayFast checkout error:", err);
    return NextResponse.json({ message: "Failed to initiate PayFast", error:
(err as Error).message }, { status: 500 });
  }
}
```

Explanation:

This PayFast integration demonstrates a comprehensive payment processing implementation for the South African market. The cart validation ensures empty carts cannot proceed to payment, preventing unnecessary payment gateway calls. The total calculation iterates through cart items, treating each as quantity 1 while accumulating the total amount and collecting item names for the payment description. The payment data structure includes all required PayFast fields: merchant credentials, URLs for payment flow control, user information, and transaction details. The signature generation process is critical for security - it creates a query string from filtered payment data (excluding empty values), URL encodes parameters with space replacement, appends the passphrase, and generates an MD5 hash. This cryptographic signature prevents payment tampering and ensures PayFast can verify the request authenticity. The URL construction uses the qs library for proper parameter encoding and supports both sandbox and production environments through conditional URL selection. The notify_url enables server-to-server payment confirmations, while return_url and cancel_url handle user redirection scenarios. This implementation supports real-world payment processing with proper error handling and security measures.

Physical Code Explanation:

Lines 1-3: Import Next.js request/response objects, `qs` for query string handling, and `md5` for signature generation. Lines 5-8: Environment variable extraction for PayFast credentials - `MERCHANT_ID`, `MERCHANT_KEY`, `PASSPHRASE` from process.env, and `SANDBOX = true` for testing mode. Line 10: `export async function POST(req: NextRequest)` defines async POST handler. Lines 12-15: Extract cart from request body with `const { cart } = await req.json()`, validate cart exists and has items, return 400 error if empty. Lines 17-21: Dummy user object for sandbox testing with `displayName` and `email` properties. Lines 23-29: Cart total calculation - initialize `totalAmount = 0` and empty `itemNames` array, use `forEach()` to iterate cart items, add each `item.price` to total with `Number()` conversion, push `item.name` to names array. Lines 31-41: PayFast payment data object construction with required fields - merchant credentials, return/cancel/notify URLs using

`process.env.NEXT_PUBLIC_BASE_URL`, user details, formatted amount with `toFixed(2)`, joined item names with `join(", ")`. Lines 43-47: Signature generation query string - `Object.entries(paymentData)` converts to key-value pairs, `filter()` removes empty values, `map()` creates "key=value" format with `encodeURIComponent()` and space replacement `replace(/%20/g, "+")`, `join("&")` creates final query string. Lines 49-55: Signature creation - append passphrase to query string, generate MD5 hash with `md5(stringToHash).toString()`, add signature to payment data. Lines 57-61: Final URL construction - use `qs.stringify()` for proper encoding, conditional URL based on `SANDBOX` flag, return PayFast URL in JSON response.

7 Database Schema (/src/types/item.ts)

7.1 Firestore Collections Structure

```typescript
// Item document - represents marketplace listings (/src/types/item.ts)
export type ItemStatus = "pending" | "for-sale" | "draft" | "sold" |
"withdrawn" | "collected";
export type ItemCondition = "new" | "excellent" | "used" | "fair" | "poor";
export type ItemCategory = "books" | "electronics" | "clothing" | "notes" |
"stationery" | "other";

export interface Item {
  id: string;                    // Firestore document ID
  title: string;                 // Item title (searchable)
  collectionAddress: string;     // Campus pickup location
  description: string;           // Detailed description
  price: number;                 // Price in cents (avoids float precision)
  condition: ItemCondition;      // Item condition state
  category: ItemCategory;        // Item category
  sellerId: string;              // Reference to user who listed item
  sellerEmail?: string;          // Seller's email for contact
  buyerId?: string;              // Buyer ID when sold
  buyerEmail?: string;           // Buyer's email when sold
  status: ItemStatus;            // Item lifecycle status
  updatedAt?: string;            // Last modification time (ISO string)
  postedAt?: string;             // Listing creation time (ISO string)
  images?: string[];             // Firebase Storage URLs array
}

// Cart document structure (from /src/app/cart/page.tsx)
interface CartItem {
  cartItemId: string;            // Unique cart item identifier
  cartId: string;                // Reference to cart
  itemId: string;                // Reference to item document
  quantity: number;              // Item quantity in cart
}

// User document - Firebase Auth integration
interface UserDocument {
  uid: string;                   // Firebase Auth UID
```

```
  email: string;                  // User's email address
  emailVerified: boolean;         // Email verification status
  displayName?: string;           // User's display name
  // Admin status managed via Firebase Custom Claims
}
```

Explanation:

This database schema represents a carefully architected data model optimized for marketplace operations and Firebase/Firestore characteristics. The UserDocument structure balances profile functionality with security - the UID directly maps to Firebase Authentication for seamless integration, while the role field enables sophisticated access control without requiring separate permissions collections. The email verification tracking supports security policies that require verified accounts for certain actions. The ItemDocument design incorporates several marketplace-specific optimizations: price storage in cents eliminates floating-point precision errors common in financial applications, the status field implements a complete item lifecycle (draft→for-sale→sold→collected) enabling complex business logic, and the sellerId reference creates efficient relationships for seller dashboards and item management. The image URLs array supports multiple product photos stored in Firebase Storage with proper access controls. The CartDocument implements a denormalized approach storing cart items directly within user documents, optimizing for read performance since carts are frequently accessed but relatively small. The timestamp fields throughout enable sophisticated analytics, audit trails, and chronological sorting. This schema design supports advanced features like item search indexing, automated status transitions, and real-time collaborative editing while maintaining optimal query performance characteristics.

Physical Code Explanation:

Lines 1-3: TypeScript type definitions using `export type` - `ItemStatus` with union of string literals using pipe `|` operator, `ItemCondition` and `ItemCategory` enums with specific allowed values. Lines 5-23: `Item` interface definition using `export interface` - `id: string` for document identifier, `title: string` for searchable item name, `collectionAddress: string` for pickup location, `price: number` with comment explaining cents storage, enum types for `condition` and `category`, string references for `sellerId`, optional properties using `?` syntax for `sellerEmail`, `buyerId`, `buyerEmail`, `status` using the defined enum, optional timestamp strings, optional `images` array of Firebase Storage URLs. Lines 25-30: `CartItem` interface with required properties for cart functionality - unique identifiers and quantity tracking. Lines 32-38: `UserDocument` interface matching Firebase Auth structure with UID, email, verification status, optional display name, and comment about admin claims management.

8 Core Components (/src/components/ItemCard.tsx)

8.1 Item Display Component (from SearchClient)

```typescript
// Item display logic from search-client.tsx
{items.map((item) => {
  const addressLines = item.collectionAddress?.split('\n') || [''];
```

```
  return (
    <Card key={item.id} className="w-full max-w-xs shadow-lg">
      <CardContent className="p-0 flex flex-col h-full">
        <div className="h-50 relative bg-sky-50 text-zinc-400 flex flex-col
justify-center items-center">
          {!!item.images?.[0] ? (
            <Image
              fill
              className="object-contain"
              src={imageUrlFormatter(item.images[0])}
              alt=""
            />
          ) : (
            <>
              <BookMarked />
              <small>No Image</small>
            </>
          )}
        </div>

        <div className="flex flex-col gap-2 p-5 flex-1">
          <div className="flex items-center gap-2 overflow-hidden">
            <p className="font-semibold truncate">{item.title}</p>
            <ItemConditionBadge
              condition={item.condition}
              className="capitalize text-base flex-shrink-0 text-xs"
            />
          </div>
          <p className="text-sm text-zinc-500">{addressLines}</p>
        </div>

        <div className="flex flex-col gap-2 p-5 pt-0">
          <p className="text-2xl font-
bold">R{numeral(item.price).format("0,0")}</p>
          <Button asChild>
            <Link href={`/item/${item.id}`}>View item</Link>
          </Button>
        </div>
      </CardContent>
    </Card>
  );
})}
```

Explanation:

This item display implementation from the SearchClient component demonstrates sophisticated marketplace UI patterns using modern React and UI library components. The Card component from Radix UI provides accessible, responsive container styling with consistent elevation and spacing. The image handling logic implements robust fallback behavior - if an item has images, it displays the first one using Next.js optimized Image component with `fill` and `object-contain` properties for proper aspect ratio handling. For

items without images, it shows a placeholder with a BookMarked icon and "No Image" text, maintaining consistent card heights. The price formatting uses the numeral library with "0,0" format for thousands separators (R1,000 instead of R1000), providing cleaner number display. The ItemConditionBadge component provides consistent condition styling across the application. The collection address handling splits multi-line addresses and displays them appropriately. The component structure uses flexbox layout (`flex flex-col`) to ensure consistent card heights with the content area expanding (`flex-1`) and action area fixed at bottom. The navigation uses Next.js Link component wrapped in a Button for proper routing and accessibility. This pattern enables consistent item presentation across search results, category pages, and recommendation systems.

Physical Code Explanation:

Line 1: `{items.map((item) => {` starts JavaScript map function to iterate over items array. Line 2: `const addressLines = item.collectionAddress?.split('\n') || [''];` uses optional chaining `?.` to safely split address by newlines, with fallback to array containing empty string. Line 4: `return (` begins JSX return statement. Line 5: `<Card key={item.id} className="w-full max-w-xs shadow-lg">` creates Card component with unique React `key` prop using item ID, Tailwind classes for full width, max width constraint, and shadow styling. Line 6: `<CardContent className="p-0 flex flex-col h-full">` creates card content container with zero padding, flexbox column layout, and full height. Lines 7-8: Comment indicating omitted lines containing image display, pricing, condition badges, and other card content. Line 9: `</Card>` closes the card component. Line 10: `});` closes the map function. Line 11: `)}` closes the JSX expression.

9 Search Functionality (/src/app/search-client.tsx)

9.1 Search Implementation with Filtering

```typescript
export default function SearchClient({ searchParams }: SearchClientProps) {
  const params = use(searchParams);
  const auth = useAuth();

  const [items, setItems] = useState<Item[]>([]);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);
  const [loading, setLoading] = useState(false);

  const minPrice = parseInt(params.minPrice || "") || null;
  const maxPrice = parseInt(params.maxPrice || "") || null;
  const condition = params.condition || null;
  const searchTerm = params.search || null;
  const sort = params.sort || "newest";
  const category = params.category || "all";

  const fetchItems = useCallback(
    async (pageNum: number, append: boolean = false) => {
      if (auth === undefined) return;

      setLoading(true);
```

```
      let token: string | null = null;
      if (auth?.currentUser) {
        token = await auth.currentUser.getIdToken();
      }

      const headers: HeadersInit = {
        "Content-Type": "application/json",
      };
      if (token) {
        headers["Authorization"] = `Bearer ${token}`;
      }

      try {
        const response = await fetch("/api/items/list", {
          method: "POST",
          headers,
          body: JSON.stringify({
            page: pageNum,
            pageSize: 10,
            minPrice,
            maxPrice,
            condition,
            status: ["for-sale"],
            searchTerm,
            sort,
            category,
          }),
        });

        const result = await response.json();
        setItems((prev) => (append ? [...prev, ...result.items] :
result.items));
        setHasMore(pageNum < result.totalPages);
      } catch (err) {
        console.error("Fetch error:", err);
        setHasMore(false);
      } finally {
        setLoading(false);
      }
    },
    [auth, minPrice, maxPrice, condition, searchTerm, sort, category]
  );
}
```

Explanation:

This search implementation demonstrates advanced performance optimization and user experience patterns essential for responsive web applications. The debouncing mechanism prevents excessive API calls during rapid typing, reducing server load and database query costs while improving perceived performance. The 300ms delay represents an optimal

balance - long enough to avoid unnecessary requests during active typing, but short enough to feel responsive to users. The URL state management approach makes search queries shareable and enables proper browser navigation behavior (back/forward buttons work correctly). The encodeURIComponent ensures special characters in search terms don't break URL formatting. The controlled input pattern with local state provides immediate visual feedback while the debounced function handles actual search execution. This architecture supports advanced features like search suggestions, category filtering, and search analytics while maintaining excellent user experience. The placeholder text specifically mentions relevant campus items, providing contextual guidance and improving search quality. The focus styling enhances accessibility and provides clear visual feedback for keyboard navigation users.

Physical Code Explanation:

Line 1: `export default function SearchClient({ searchParams }: SearchClientProps) {` declares component function with destructured props and TypeScript type annotation. Line 2: `const params = use(searchParams);` uses React's `use` hook to unwrap the searchParams promise. Line 3: `const auth = useAuth();` gets authentication context. Lines 5-8: State initialization with `useState` - `items` array for search results, `page` number for pagination, `hasMore` boolean for infinite scroll, `loading` for request state. Lines 10-15: Parameter extraction and parsing - `parseInt(params.minPrice || "") || null` converts string to number with fallback to null, similar pattern for maxPrice, direct assignment for condition/searchTerm/sort/category with defaults. Lines 17-20: `fetchItems` function declared with `useCallback` for performance - takes `pageNum` and optional `append` boolean, checks auth state with early return if undefined.

10 PayFast Security Implementation (/src/app/api/payfast/checkout/route.ts)

10.1 PayFast Security: Signature Generation & Validation

10.1.1 Signature Generation in Checkout Route (/src/app/api/payfast/checkout/route.ts)

```typescript
// Create query string for signature generation
const queryStringForSignature = Object.entries(paymentData)
  .filter(([, value]) => value !== null && value !== undefined && value !== "")
  .map(([key, value]) => `${key}=${encodeURIComponent(String(value)).replace(/%20/g, "+")}`)
  .join("&");

// Add passphrase
const stringToHash = `${queryStringForSignature}&passphrase=${PASSPHRASE}`;

// Generate signature
const signature = md5(stringToHash).toString();
paymentData.signature = signature;

// Create final query string for redirect URL
```

```
const finalQueryString = qs.stringify(paymentData, { encode: true });

const payfastUrl = SANDBOX
  ? `https://sandbox.payfast.co.za/eng/process?${finalQueryString}`
  : `https://www.payfast.co.za/eng/process?${finalQueryString}`;
```

10.1.2 Signature Validation in Notify Route (/src/app/api/payfast/notify/route.ts)

```typescript
// Remove signature from parameters while preserving original order
let baseParamString = rawBody
  .replace(/(^|&)signature=[^&]*/,'')  // Remove signature parameter
  .replace(/^&/, '')                   // Clean leading &
  .trim();

// Append passphrase if configured
let stringToHash = baseParamString;
if (PASSPHRASE && PASSPHRASE.length > 0) {
  stringToHash += `&passphrase=${PASSPHRASE}`;
}

// Generate signature from received parameters
const generatedSignature = md5(stringToHash).toString();

// Validate against PayFast's signature
if (generatedSignature !== receivedSignature) {
  console.error("Signature mismatch – investigate");
  return new NextResponse("Signature verification failed", { status: 400 });
}
```

Explanation:

PayFast uses a two-way signature system for complete payment security.

Checkout Route:

Your server generates a signature when creating payment requests to prove authenticity to PayFast - this prevents tampering with payment amounts, merchant details, or redirect URLs before reaching PayFast's servers.

Notify Route:

PayFast generates signatures when sending ITN (Instant Transaction Notification) confirmations back to your server - your server validates these signatures to ensure payment notifications are legitimate and haven't been spoofed by attackers. The signature generation process filters empty values, preserves parameter order (critical for PayFast compatibility), URL-encodes special characters, and creates MD5 hashes with the merchant passphrase. This dual-signature approach ensures end-to-end payment integrity: outbound requests are

authenticated by your signature, and inbound confirmations are validated against PayFast's signature. The notify route implementation preserves the original parameter order received from PayFast, which is essential since PayFast's signature validation requires exact parameter matching including order and encoding.

Physical Code Explanation:

Lines 1-6: Signature removal from raw body - `rawBody.replace(/(^|&)signature=[^&]*/,'')` uses regex to match signature parameter at start `^` or after ampersand `&`, captures signature value until next ampersand `[^&]*`, removes entire match, `.replace(/^&/, '')` cleans leading ampersand left after removal, `.trim()` removes whitespace. Lines 8-12: Passphrase handling - `stringToHash` starts with base parameter string, `if (PASSPHRASE && PASSPHRASE.length > 0)` checks passphrase exists and is non-empty, `stringToHash += \`&passphrase=${PASSPHRASE}\`` appends passphrase using template literal. Line 15: `const generatedSignature = md5(stringToHash).toString()` creates MD5 hash of final string and converts to string representation. Lines 18-21: Signature validation - `if (generatedSignature !== receivedSignature)` compares generated hash with PayFast's signature, logs error for debugging, returns 400 Bad Request with descriptive error message using `new NextResponse()`.

11 Form Validation (/src/validation/itemSchema.ts)

11.1 Item Validation Schema with Zod

```typescript
import { z } from "zod";

export const itemDataSchema = z.object({
  title: z.string().min(1, "Title is required"),
  collectionAddress: z.string().min(1, "Collection address must have a
value"),
  description: z.string().min(1, "Description is required"),
  price: z.coerce.number().min(1, "Price is required"),
  status: z.enum(["draft", "pending", "for-sale", "sold", "withdrawn",
"collected"]),
  sellerId: z.string(),
  condition: z.enum(["new", "excellent", "used", "fair", "poor"]),
  category: z.enum([
    "books",
    "electronics",
    "clothing",
    "notes",
    "stationery",
    "other",
  ]),
});

export const itemImagesSchema = z.object({
  images: z
    .array(
```

```
      z.object({
        id: z.string(),
        url: z.string(),
        file: z.instanceof(File).optional(),
      })
    )
    .min(1, "At least one image is required"),
});

export const itemSchema = itemDataSchema.and(itemImagesSchema);
```

Explanation:

This validation implementation showcases modern form handling combining Zod's runtime type safety with React Hook Form's performance optimizations. The schema design reflects marketplace-specific business rules: title length constraints optimize search indexing and display consistency, description requirements ensure sufficient product information for buyer decisions, and price limits prevent obviously incorrect listings while accommodating high-value items like electronics or furniture. The enum validations for category and condition create consistent data taxonomy enabling reliable filtering and search functionality. The Zod integration provides compile-time TypeScript inference, meaning form data types are automatically derived from the schema, eliminating type annotation duplication and ensuring schema-form consistency. The React Hook Form resolver pattern enables real-time validation feedback without performance-killing re-renders, as validation only occurs on field blur or form submission. The default values establish sensible form state preventing empty form confusion while the 'other' category default accommodates diverse marketplace items. This validation architecture supports complex features like conditional validation, custom validators, and multi-step form workflows while maintaining excellent developer experience and runtime performance characteristics.

Physical Code Explanation:

Line 1: `import { z } from "zod"` imports Zod validation library. Lines 3-15: `export const itemDataSchema = z.object({` creates Zod object schema with validation rules - `z.string().min(1, "Title is required")` requires non-empty string for title, `z.string().min(1, "Collection address must have a value")` validates collection address, `z.coerce.number().min(1, "Price is required")` converts input to number and validates minimum value, `z.enum([...])` creates enumeration validation for status, condition, and category fields with specific allowed values. Lines 17-24: `export const itemImagesSchema = z.object({` defines image validation schema with `z.array()` for image array, `z.object()` for individual image structure, `z.string()` for id and url fields, `z.instanceof(File).optional()` for optional file object, `.min(1, "At least one image is required")` ensures at least one image. Line 26: `export const itemSchema = itemDataSchema.and(itemImagesSchema)` combines both schemas using `.and()` method for complete form validation.

12 Project Structure

```
src/
├── app/                        # Next.js App Router pages
```

```
│   ├── page.tsx               # Home page with item listings
│   ├── search-client.tsx      # Search functionality component
│   ├── layout.tsx             # Root layout with providers
│   ├── globals.css            # Global styles
│   ├── admin/                 # Admin dashboard and management
│   │   └── page.tsx           # Admin analytics dashboard
│   ├── auth/                  # Authentication flow pages
│   │   └── verify-email/      # Email verification
│   ├── cart/                  # Shopping cart system
│   │   └── page.tsx           # Cart page with PayFast integration
│   ├── debug/                 # Development debugging tools
│   ├── how/                   # Help and how-to pages
│   ├── item/                  # Individual item pages
│   │   └── [itemId]/          # Dynamic item detail pages
│   ├── item-search/           # Advanced search functionality
│   │   └── filters-form.tsx   # Search filters component
│   ├── login/                 # User authentication
│   │   └── page.tsx           # Login form
│   ├── privacy/               # Privacy policy pages
│   ├── profile/               # User profile management
│   │   └── page.tsx           # Profile dashboard
│   ├── register/              # User registration
│   │   ├── page.tsx           # Registration form
│   │   └── actions.ts         # Server actions for registration
│   ├── reset-password/        # Password reset flow
│   ├── success/               # Payment success pages
│   ├── terms/                 # Terms and conditions
│   └── api/                   # Backend API routes
│       ├── admin/             # Admin-only endpoints
│       ├── auth/              # Authentication endpoints
│       ├── cart/              # Shopping cart APIs
│       ├── debug/             # Development/testing endpoints
│       ├── items/             # Item management APIs
│       ├── payfast/           # PayFast payment processing
│       │   ├── checkout/      # Payment initiation
│       │   └── notify/        # Payment notifications (ITN)
│       ├── payments/          # Payment history and management
│       ├── purchases/         # Purchase tracking
│       ├── refresh-token/     # Token refresh endpoints
│       └── users/             # User management APIs
├── components/                 # Reusable React components
│   ├── ui/                    # Base UI components (Radix UI)
│   ├── AddToCartButton.tsx    # Shopping cart integration
│   ├── auth-buttons.tsx       # Authentication UI components
│   ├── collected-button.tsx   # Item collection status
│   ├── CookieNotice.tsx       # GDPR cookie compliance
│   ├── email-buyer-button.tsx # Buyer contact functionality
│   ├── email-seller-button-profile.tsx # Seller contact
│   ├── get-items-admin.tsx    # Admin item management
│   ├── Input.tsx              # Form input components
│   ├── item-category-badge.tsx # Category display badges
│   ├── item-condition-badge.tsx # Condition status badges
```

```
│   ├── item-form.tsx          # Item creation/editing form
│   ├── item-status-badge.tsx  # Item status indicators
│   ├── multi-image-uploader.tsx # Drag & drop image uploads
│   ├── PhotoUpload.tsx        # Single photo upload component
│   └── SessionManager.tsx     # Session timeout management
├── context/                    # React context providers
│   ├── auth.tsx               # Authentication state management
│   └── actions.ts             # Server actions for auth
├── firebase/                   # Firebase configuration
│   ├── client.ts              # Client-side Firebase SDK
│   └── server.ts              # Server-side Admin SDK
├── hooks/                      # Custom React hooks
│   └── useSessionTimeout.ts   # Session management hook
├── lib/                        # Utility functions
│   ├── utils.ts               # General utilities
│   ├── userUtils.ts           # User-related helpers
│   ├── sessionUtils.ts        # Session management
│   ├── itemManagement.ts      # Item operations
│   ├── imageUrlFormatter.ts   # Image URL processing
│   └── cartCleanup.ts         # Cart maintenance
├── types/                      # TypeScript definitions
│   ├── index.ts               # Main type definitions
│   ├── item.ts                # Item-related types
│   ├── itemStatus.ts          # Item status enums
│   ├── itemCondition.ts       # Item condition types
│   ├── itemCategory.ts        # Category definitions
│   ├── CreateItemResponse.ts  # API response types
│   ├── GetItemResponse.ts     # Single item response
│   └── GetItemsResponse.ts    # Items list response
├── validation/                 # Form validation schemas
│   ├── itemSchema.ts          # Item creation validation
│   └── registerUser.ts        # User registration validation
└── middleware.ts               # Route protection and auth
```

Explanation:

This project structure follows Next.js 15 App Router conventions optimized for a modern marketplace application. The `/app` directory contains all routes using the new file-based routing system, with `page.tsx` files defining route components and `layout.tsx` files providing shared layouts. The API routes in `/app/api` implement server-side functionality using Next.js API routes with proper organization by feature (items, admin, payments). The components directory separates UI components with a dedicated `/ui` folder for base components and feature-specific components at the root level. The context system manages global state including authentication and provides server actions for secure operations. Firebase configuration is split between client and server implementations for optimal security and performance. The lib directory contains utility functions organized by domain, supporting code reusability and maintainability. Type definitions in `/types` provide comprehensive TypeScript support across the application, while validation schemas ensure data integrity at form submission points. This structure supports scalable development with clear separation of concerns and follows modern React and Next.js best practices.

Physical Code Explanation:

The directory tree uses standard filesystem notation - `/` indicates directories, ` ├──` and ` └──` show tree structure relationships, `#` comments explain directory purposes. `src/app/` follows Next.js 13+ App Router convention where each folder can contain `page.tsx` (route component), `layout.tsx` (shared layout), and `loading.tsx` (loading UI). `[itemId]/` uses bracket notation for dynamic routes where `itemId` becomes a URL parameter. `api/` folder contains Next.js API routes where each `route.ts` file exports HTTP method handlers (GET, POST, etc.). Components are organized flat in `/components` with `/ui` subfolder for reusable base components. `/context` contains React Context providers for global state management. `/firebase` splits client and server SDK configurations for security. `/hooks` contains custom React hooks following `use` naming convention. `/lib` contains utility functions organized by domain. `/types` contains TypeScript type definitions with descriptive filenames. `/validation` contains Zod schemas for form validation. Root-level `middleware.ts` implements Next.js middleware for route protection.

13 Admin Dashboard & Analytics (/src/app/admin/page.tsx)

13.1 Real Admin Dashboard Implementation

```typescript
export default function AdminDashboard() {
  const auth = useAuth();
  const [loading, setLoading] = useState(true);
  const [statusCounts, setStatusCounts] = useState<Record<string,
number>>({});
  const [priceBuckets, setPriceBuckets] = useState<{ range: string; count:
number }[]>([]);
  const [totalUsers, setTotalUsers] = useState(0);
  const [activeUsers, setActiveUsers] = useState(0);
  const [totalItems, setTotalItems] = useState(0);
  const [itemsForSale, setItemsForSale] = useState(0);
  const [totalTurnover, setTotalTurnover] = useState(0);

  useEffect(() => {
    const fetchData = async () => {
      const user = auth?.currentUser;
      if (!user) return;

      const token = await user.getIdToken();

      try {
        // Test admin access first
        const statusRes = await fetch("/api/admin/status", {
          headers: { Authorization: `Bearer ${token}` }
        });

        if (!statusRes.ok) {
          throw new Error("Admin access denied");
        }
```

```
        // Fetch items data
        const itemsRes = await fetch("/api/items/list", {
          method: "POST",
          headers: {
            Authorization: `Bearer ${token}`,
            "Content-Type": "application/json",
          },
          body: JSON.stringify({ page: 1, pageSize: 1000 }),
        });

        const itemsData = await itemsRes.json();
        if (!itemsRes.ok || !itemsData.success) throw new Error("Failed to
fetch items");

        // Calculate analytics from items data
        const items = itemsData.items;
        setTotalItems(items.length);
        setItemsForSale(items.filter(item => item.status === 'for-
sale').length);

        // Status distribution
        const statusCount = items.reduce((acc, item) => {
          acc[item.status] = (acc[item.status] || 0) + 1;
          return acc;
        }, {});
        setStatusCounts(statusCount);

        // Price analysis
        const buckets = [
          { range: "R0-R50", min: 0, max: 5000 },
          { range: "R50-R200", min: 5000, max: 20000 },
          { range: "R200-R500", min: 20000, max: 50000 },
          { range: "R500+", min: 50000, max: Infinity }
        ];

        const bucketCounts = buckets.map(bucket => ({
          range: bucket.range,
          count: items.filter(item => item.price >= bucket.min && item.price <
bucket.max).length
        }));
        setPriceBuckets(bucketCounts);

      } catch (error) {
        console.error("Admin fetch error:", error);
        toast.error("Failed to load admin data");
      } finally {
        setLoading(false);
      }
    };

    fetchData();
  }, [auth]);
```

```
  return (
    <div className="p-6 space-y-6">
      {/* Key Metrics Cards */}
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        <div className="bg-white p-4 rounded-lg shadow">
          <h3 className="text-sm font-medium text-gray-500">Total Items</h3>
          <p className="text-2xl font-bold">{totalItems}</p>
        </div>
        <div className="bg-white p-4 rounded-lg shadow">
          <h3 className="text-sm font-medium text-gray-500">Items For
Sale</h3>
          <p className="text-2xl font-bold">{itemsForSale}</p>
        </div>
        <div className="bg-white p-4 rounded-lg shadow">
          <h3 className="text-sm font-medium text-gray-500">Total Users</h3>
          <p className="text-2xl font-bold">{totalUsers}</p>
        </div>
        <div className="bg-white p-4 rounded-lg shadow">
          <h3 className="text-sm font-medium text-gray-500">Revenue</h3>
          <p className="text-2xl font-bold">R{numeral(totalTurnover /
100).format('0,0')}</p>
        </div>
      </div>

      {/* Charts Section */}
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Item Status Distribution */}
        <div className="bg-white p-4 rounded-lg shadow">
          <h3 className="text-lg font-semibold mb-4">Items by Status</h3>
          <ResponsiveContainer width="100%" height={300}>
            <PieChart>
              <Pie
                data={Object.entries(statusCounts).map(([status, count]) => ({
status, count }))}
                dataKey="count"
                nameKey="status"
                cx="50%"
                cy="50%"
                outerRadius={80}
                label
              >
                {Object.entries(statusCounts).map(([status], index) => (
                  <Cell key={`cell-${index}`} fill={STATUS_COLORS[status] ||
'#8884d8'} />
                ))}
              </Pie>
              <Tooltip />
              <Legend />
            </PieChart>
          </ResponsiveContainer>
        </div>
```

```
        {/* Price Distribution */}
        <div className="bg-white p-4 rounded-lg shadow">
          <h3 className="text-lg font-semibold mb-4">Price Distribution</h3>
          <ResponsiveContainer width="100%" height={300}>
            <BarChart data={priceBuckets}>
              <CartesianGrid strokeDasharray="3 3" />
              <XAxis dataKey="range" />
              <YAxis />
              <Tooltip />
              <Bar dataKey="count" fill="#3b82f6" />
            </BarChart>
          </ResponsiveContainer>
        </div>
      </div>
    </div>
  );
}
```

Explanation:

This admin dashboard implementation represents the actual command center for marketplace operations, using real Recharts components for data visualization. The dashboard fetches live data from your existing API endpoints and calculates meaningful analytics including item status distribution, price analysis, and user metrics. The implementation uses your actual admin authentication flow via `/api/admin/status` and leverages the existing items API for comprehensive data analysis. The Recharts integration provides interactive charts that automatically handle responsive design and touch interactions for mobile admin access.

Physical Code Explanation:

Line 1: `"use client"` enables browser-side React features. Lines 3-15: Import statements - React hooks (`useEffect`, `useState`), authentication context (`useAuth`), toast notifications (`toast`), number formatting (`numeral`), and Recharts components (`BarChart`, `Bar`, `XAxis`, `YAxis`, `CartesianGrid`, `Tooltip`, `PieChart`, `Pie`, `Cell`, `Legend`, `ResponsiveContainer`). Lines 17-22: `Item` interface definition with `id` string, `price` number, `status` with specific enum values, and optional `sellerId`. Lines 24-28: `User` interface with `id`, `email`, and `isActive` properties. Lines 30-37: `STATUS_COLORS` constant object mapping status strings to hex color codes for chart visualization. Lines 39-42: Component function declaration with auth context initialization and loading state. Lines 44-46: State variables using `useState` - `setItems` and `setUsers` for data storage (note the destructuring ignores the getter), `statusCounts` for status distribution tracking.

14 Shopping Cart System (/src/app/cart/page.tsx)

14.1 Real Cart Implementation with VAT

```typescript
export default function CartPage() {
```

```
  const auth = useAuth();
  const [cartItems, setCartItems] = useState<CartItem[]>([]);
  const [itemsData, setItemsData] = useState<ItemData[]>([]);
  const [loading, setLoading] = useState(true);
  const [agreed, setAgreed] = useState(false);

  const handleCheckout = async () => {
    if (!agreed) {
      toast.error("Please agree to the terms and conditions");
      return;
    }

    setLoading(true);

    try {
      // Prepare cart data for PayFast
      const cartForCheckout = cartItems.map(cartItem => {
        const item = itemsData.find(i => i.id === cartItem.itemId);
        return item ? {
          id: item.id,
          name: item.title,
          price: item.price,
          quantity: cartItem.quantity || 1
        } : null;
      }).filter(Boolean);

      // Calculate total with VAT (15% South African VAT)
      const totalAmount = cartForCheckout.reduce((sum, item) =>
        sum + (item ? item.price * item.quantity : 0), 0
      );
      const totalWithVat = totalAmount * 1.15;

      const token = await auth.currentUser.getIdToken();

      const res = await fetch("/api/payfast/checkout", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`,
        },
        body: JSON.stringify({ cart: cartForCheckout, totalAmount:
totalWithVat }),
      });

      const data = await res.json();

      if (data.url) {
        // Store cart state before payment for potential recovery
        localStorage.setItem('cartBeforePayment', JSON.stringify({ cartItems,
itemsData }));
        window.location.href = data.url; // Redirect to PayFast
      } else {
```

```
        toast.error("Error initiating payment: " + (data.message || ""));
      }
    } catch (err) {
      console.error(err);
      toast.error("Failed to initiate payment");
    } finally {
      setLoading(false);
    }
  };

  // Calculate totals for display
  const subtotal = cartItems.reduce((sum, cartItem) => {
    const item = itemsData.find(i => i.id === cartItem.itemId);
    return sum + (item ? item.price * (cartItem.quantity || 1) : 0);
  }, 0);

  const vatAmount = subtotal * 0.15; // 15% VAT
  const total = subtotal + vatAmount;

  return (
    <div className="min-h-screen flex justify-center items-start px-4 mb-8">
      <div className="bg-white shadow-xl rounded-2xl w-full max-w-2xl p-8">
        <h2 className="text-3xl font-bold mb-6 text-gray-800 text-center">Your
Cart</h2>

        {loading ? (
          <p className="text-gray-500 text-center py-10">Loading cart...</p>
        ) : itemsData.length === 0 ? (
          <p className="text-gray-500 text-center py-10">Your cart is
empty.</p>
        ) : (
          <>
            {/* Cart Items List */}
            <ul className="divide-y divide-gray-200">
              {cartItems.map(cartItem => {
                const item = itemsData.find(i => i.id === cartItem.itemId);
                if (!item) return null;

                return (
                  <li key={cartItem.cartItemId} className="py-4 flex items-
center space-x-4">
                    <div className="h-16 w-16 rounded-lg overflow-hidden">
                      {item.images?.[0] ? (
                        <Image
                          src={imageUrlFormatter(item.images[0])}
                          alt={item.title}
                          width={64}
                          height={64}
                          className="object-cover w-full h-full"
                        />
                      ) : (
```

```
                      <div className="w-full h-full bg-gray-200 flex items-
center justify-center">
                        <span className="text-gray-400 text-xs">No
Image</span>
                      </div>
                    )}
                  </div>

                  <div className="flex-1 min-w-0">
                    <h4 className="text-sm font-medium text-gray-900
truncate">{item.title}</h4>
                    <p className="text-sm text-gray-500">Condition:
{item.condition}</p>
                    <p className="text-sm font-medium">R{numeral(item.price
/ 100).format('0,0.00')}</p>
                  </div>

                  <div className="flex items-center space-x-2">
                    <span className="text-sm text-gray-500">Qty:
{cartItem.quantity || 1}</span>
                  </div>
                </li>
              );
            })}
          </ul>

          {/* Order Summary */}
          <div className="mt-6 pt-6 border-t border-gray-200">
            <div className="flex justify-between text-sm">
              <span>Subtotal</span>
              <span>R{numeral(subtotal / 100).format('0,0.00')}</span>
            </div>
            <div className="flex justify-between text-sm">
              <span>VAT (15%)</span>
              <span>R{numeral(vatAmount / 100).format('0,0.00')}</span>
            </div>
            <div className="flex justify-between text-lg font-bold mt-2">
              <span>Total</span>
              <span>R{numeral(total / 100).format('0,0.00')}</span>
            </div>
          </div>

          {/* Terms Agreement & Checkout */}
          <div className="mt-6">
            <label className="flex items-center space-x-2">
              <input
                type="checkbox"
                checked={agreed}
                onChange={(e) => setAgreed(e.target.checked)}
                className="rounded border-gray-300"
              />
              <span className="text-sm text-gray-600">
```

```
                I agree to the terms and conditions
              </span>
            </label>

            <Button
              onClick={handleCheckout}
              disabled={!agreed || loading}
              className="w-full mt-4 bg-blue-600 text-white py-3 rounded-lg
hover:bg-blue-700 disabled:opacity-50"
            >
              {loading ? "Processing..." : "Proceed to Payment"}
            </Button>
          </div>
        </>
      )}
    </div>
  </div>
 );
}
```

Explanation:

This cart implementation provides a complete South African e-commerce experience with proper VAT calculations and PayFast integration. The component fetches cart data from your existing `/api/cart` endpoint and displays items with quantity management. The VAT calculation correctly applies 15% South African VAT, and the checkout process integrates with your existing PayFast implementation. The cart state persistence in localStorage enables recovery after payment processing.

Physical Code Explanation:

Line 1: `"use client"` enables browser-side React features. Lines 3-9: Import React hooks (`useEffect`, `useState`), Next.js components (`Link`, `Image`), notification system (`toast`), custom Button component, image formatter utility, and authentication context. Lines 11-16: `CartItem` interface defines cart item structure with `cartItemId`, `cartId`, `itemId`, and `quantity` properties. Lines 18-25: `ItemData` interface defines item details structure with `id`, `title`, `price`, `images` array, `condition`, and `sellerId`. Lines 27-31: Component state initialization - `cartItems` array for cart data, `itemsData` array for item details, `loading` boolean for loading state, `agreed` boolean for terms acceptance. Lines 33-54: `useEffect` hook for cart fetching - checks user authentication with `auth?.currentUser`, gets ID token with `await auth.currentUser.getIdToken()`, makes GET request to `/api/cart` with Authorization header, parses response and updates state arrays. Lines 56-60: Subtotal calculation using `reduce()` - finds matching cart item for each item with `cartItems.find()`, extracts quantity with fallback to 1, multiplies `item.price * quantity` and accumulates sum. Lines 61-62: VAT and total calculation - applies 15% South African VAT with `subtotal * 0.15`, adds VAT to subtotal for final total. Lines 64-73: `handleDelete` function for removing cart items - authenticates user, gets token, makes DELETE request to `/api/cart/${cartItemId}`, updates local state by filtering out deleted item with `prev.filter()`.

15 Image Upload System (/src/components/multi-image-uploader.tsx)

15.1 Real Multi-Image Uploader Implementation

```typescript
export default function MultiImageUploader({
    images = [],
    onImagesChange,
    urlFormatter
}: Props){
    const uploadInputRef = useRef<HTMLInputElement | null>(null);
    const MAX_IMAGES = 3;

    const handleInputChange = (e: React.ChangeEvent<HTMLInputElement>) => {
        const files = Array.from(e.target.files || []);
        const remainingSlots = MAX_IMAGES - images.length;

        if (remainingSlots <= 0) {
            toast.info("", {
                description: `You can only upload ${MAX_IMAGES} images`,
            });
            return;
        }

        const limitedFiles = files.slice(0, remainingSlots);

        const newImages = limitedFiles.map((file, index) => ({
            id: `upload-${Date.now()}-${index}`,
            url: URL.createObjectURL(file),
            file: file
        }));

        onImagesChange([...images, ...newImages]);

        if (uploadInputRef.current) {
            uploadInputRef.current.value = "";
        }
    };

    const handleDragEnd = (result: DropResult) => {
        if (!result.destination) return;

        const reorderedImages = Array.from(images);
        const [removed] = reorderedImages.splice(result.source.index, 1);
        reorderedImages.splice(result.destination.index, 0, removed);

        onImagesChange(reorderedImages);
    };

    const removeImage = (imageId: string) => {
        const updatedImages = images.filter(img => img.id !== imageId);
```

```
        onImagesChange(updatedImages);
    };

    return (
        <div className="space-y-4">
            {/* Image Grid with Drag & Drop */}
            <DragDropContext onDragEnd={handleDragEnd}>
                <Droppable droppableId="images" direction="horizontal">
                    {(provided) => (
                        <div
                            {...provided.droppableProps}
                            ref={provided.innerRef}
                            className="flex gap-4 flex-wrap"
                        >
                            {images.map((image, index) => (
                                <Draggable key={image.id}
draggableId={image.id} index={index}>
                                    {(provided) => (
                                        <div
                                            ref={provided.innerRef}
                                            {...provided.draggableProps}
                                            className="relative group"
                                        >
                                            <div className="w-32 h-32 border-2
border-gray-300 rounded-lg overflow-hidden">
                                                <Image
                                                    src={urlFormatter ?
urlFormatter(image) : image.url}
                                                    alt={`Upload ${index +
1}`}
                                                    width={128}
                                                    height={128}
                                                    className="w-full h-full
object-cover"
                                                />
                                            </div>

                                            {/* Primary Badge */}
                                            {index === 0 && (
                                                <Badge className="absolute
top-2 left-2 text-xs">
                                                    Primary
                                                </Badge>
                                            )}

                                            {/* Drag Handle */}
                                            <div
                                                {...provided.dragHandleProps}
                                                className="absolute top-2
right-8 p-1 bg-white rounded opacity-0 group-hover:opacity-100 transition-
opacity cursor-grab"
                                            >
```

```
                                            <MoveIcon size={16} />
                                        </div>

                                        {/* Remove Button */}
                                        <button
                                            onClick={() =>
removeImage(image.id)}
                                            className="absolute top-2
right-2 p-1 bg-red-500 text-white rounded opacity-0 group-hover:opacity-100
transition-opacity"
                                        >
                                            <XIcon size={16} />
                                        </button>
                                    </div>
                                )}
                            </Draggable>
                        ))}
                        {provided.placeholder}
                    </div>
                )}
            </Droppable>
        </DragDropContext>

        {/* Upload Button */}
        {images.length < MAX_IMAGES && (
            <div>
                <input
                    ref={uploadInputRef}
                    type="file"
                    multiple
                    accept="image/*"
                    onChange={handleInputChange}
                    className="hidden"
                />
                <Button
                    type="button"
                    variant="outline"
                    onClick={() => uploadInputRef.current?.click()}
                    className="w-full h-32 border-2 border-dashed border-
gray-300 hover:border-gray-400"
                >
                    <div className="text-center">
                        <p className="text-sm text-gray-600">Click to
upload images</p>
                        <p className="text-xs text-gray-400">
                            {images.length}/{MAX_IMAGES} images uploaded
                        </p>
                    </div>
                </Button>
            </div>
        )}
    </div>
```

```
    );
}
```

Explanation:

This multi-image uploader implements a sophisticated drag-and-drop interface using `@hello-pangea/dnd` for image reordering. The component enforces a 3-image limit, provides visual feedback for primary images, and includes proper file handling with URL.createObjectURL for immediate preview. The implementation supports your existing image upload workflow while providing an intuitive user experience for marketplace listings. Additionally when a user edits an item and keeps existing images the 'url' variable keeps track of images already in Firebase storage and the 'file' variable keeps track of new images that need to be uploaded to Firebase.

Physical Code Explanation:

Line 1: `"use client"` directive for browser-side React features. Lines 3-10: Import statements - `useRef` for DOM references, drag-drop components from `@hello-pangea/dnd`, Next.js `Image` component, UI components (`Badge`, `Button`), icons (`MoveIcon`, `XIcon`), and `toast` for notifications. Lines 12-16: TypeScript type definition for `ImageUpload` object with `id` string, `url` string, and optional `file` property. Lines 18-22: Props interface defining component inputs - optional `images` array, required `onImagesChange` callback function, optional `urlFormatter` function. Lines 24-28: Component function declaration with destructured props and default empty array for images. Line 30: `useRef<HTMLInputElement | null>(null)` creates reference for hidden file input. Line 34: `const MAX_IMAGES = 3` sets upload limit constant. Lines 36-50: `handleInputChange` function - `Array.from(e.target.files || [])` converts FileList to array, `remainingSlots = MAX_IMAGES - images.length` calculates available slots, shows toast message if limit exceeded, `files.slice(0, remainingSlots)` limits file selection. Lines 52-57: New image object creation - `map()` function creates objects with unique IDs using `Date.now()` timestamp, `URL.createObjectURL(file)` creates blob URL for preview, stores original file object, calls `onImagesChange([...images, ...newImages])` to update parent state. Lines 67-74: `handleDragEnd` function for reordering - checks `result.destination` exists, creates mutable copy with `Array.from(images)`, uses `splice()` to remove and insert at new position, calls `onImagesChange(items)` with reordered array.

16 Additional Components

16.1 Add to Cart Button (/src/components/AddToCartButton.tsx)

```typescript
export default function AddToCartButton({ id }: { id: string }) {
  const router = useRouter();
  const auth = useAuth();
  const [isAdding, setIsAdding] = useState(false);

  const handleAddToCart = async () => {
    if (!auth?.currentUser) {
      router.push("/login");
      return;
```

```
    }

    setIsAdding(true);

    try {
      const token = await auth.currentUser.getIdToken();
      const res = await fetch("/api/items/actions/add-to-cart", {
        method: "POST",
        headers: {
          Authorization: `Bearer ${token}`,
          "Content-Type": "application/json",
        },
        body: JSON.stringify({ itemId: id }),
      });

      const data = await res.json();
      if (!res.ok || !data.success) {
        toast.error(data.message || "Failed to add item to cart");
      } else {
        toast.success("Item added to cart");
        router.push("/cart");
      }
    } catch (err) {
      console.error(err);
      toast.error("Failed to add item to cart");
    } finally {
      setIsAdding(false);
    }
  };

  return (
    <Button className="flex-1 w-full" onClick={handleAddToCart}
disabled={isAdding}>
      {isAdding ? "Adding..." : "Add to cart"}
    </Button>
  );
}
```

Physical Code Explanation:

Line 1: `export default function AddToCartButton({ id }: { id: string }) {` declares component with destructured props and TypeScript type annotation for item ID. Lines 2-4: Initialize hooks - `useRouter()` for navigation, `useAuth()` for authentication context, `useState(false)` for loading state with `isAdding` boolean. Lines 6-10: `handleAddToCart` async function - checks authentication with `!auth?.currentUser` using optional chaining, redirects to login with `router.push("/login")` if not authenticated, early return to prevent further execution. Lines 12-13: Set loading state with `setIsAdding(true)` to show visual feedback. Lines 15-24: API call setup - get authentication token with `await auth.currentUser.getIdToken()`, make POST request to add-to-cart endpoint with `fetch()`, set Authorization header using template literal `Bearer ${token}`, stringify body with `JSON.stringify({ itemId: id })`. Lines 26-31: Response handling - parse JSON with `await res.json()`, check for errors with `!res.ok ||

!data.success`, show error toast with fallback message, show success toast and navigate to cart on success. Lines 32-36: Error handling - catch any exceptions, log error to console, show generic error toast, `finally` block ensures loading state is reset. Lines 39-42: JSX return - Button component with flexbox styling, click handler, disabled state based on loading, conditional text content using ternary operator.

16.2 Authentication Buttons (/src/components/auth-buttons.tsx)

```typescript
export default function AuthButtons() {
  const router = useRouter();
  const auth = useAuth();
  const [dropdownOpen, setDropdownOpen] = useState(false);

  if (auth.isLoading) {
    return (
      <div className="flex items-center gap-4">
        <div className="h-4 w-20 rounded bg-gray-300 animate-pulse" />
        <div className="h-4 w-20 rounded bg-gray-300 animate-pulse" />
      </div>
    );
  }

  return (
    <div className="flex items-center gap-4">
      {auth.currentUser ? (
        <>
          {/* Sell Item */}
          <Link
            href="/profile/new"
            className="font-semibold text-secondary hover:text-accent
hover:underline underline-offset-4 transition"
          >
            Sell Item
          </Link>
          <div className="h-6 w-[1px] bg-gray-500/50" />
          <Link
            href="/cart"
            className="text-gray-700 hover:text-accent hover:underline
underline-offset-4 transition"
          >
            Cart
          </Link>

          {/* Account Dropdown */}
          <DropdownMenu open={dropdownOpen} onOpenChange={setDropdownOpen}>
            <DropdownMenuTrigger asChild>
              <div
                onClick={() => setDropdownOpen(!dropdownOpen)}
```

```
                className={`flex items-center gap-1 cursor-pointer transition
${
                  dropdownOpen ? "text-accent underline underline-offset-4" :
"text-gray-700 hover:text-accent hover:underline underline-offset-4"
                }`}
              >
                Account
                <ChevronDown className="h-4 w-4" />
              </div>
            </DropdownMenuTrigger>

            <DropdownMenuContent align="start" className="rounded-sm mt-1 min-
w-[180px] bg-white border border-gray-200 p-1">
              <DropdownMenuLabel className="px-3 py-2">
                <div className="font-
medium">{auth.currentUser.displayName}</div>
                <div className="text-xs text-gray-
700">{auth.currentUser.email}</div>
              </DropdownMenuLabel>

              <DropdownMenuSeparator />

              {auth.isAdmin && (
                <DropdownMenuItem
                  onClick={() => router.push("/admin")}
                  className="px-3 py-2 hover:bg-gray-100 cursor-pointer
rounded-sm"
                >
                  Admin Dashboard
                </DropdownMenuItem>
              )}

              <DropdownMenuItem
                onClick={() => router.push("/profile/user")}
                className="px-3 py-2 hover:bg-gray-100 cursor-pointer rounded-
sm"
              >
                My Profile
              </DropdownMenuItem>

              <DropdownMenuItem
                onClick={async () => {
                  await auth.logout();
                  router.refresh();
                }}
                className="px-3 py-2 hover:bg-gray-100 cursor-pointer rounded-
sm"
              >
                Logout
              </DropdownMenuItem>
            </DropdownMenuContent>
          </DropdownMenu>
```

```
        </>
      ) : (
        <>
          <Link href="/login" className="text-gray-700 hover:text-accent
hover:underline underline-offset-4 transition">
            Login
          </Link>
          <div className="h-6 w-[1px] bg-gray-500/50" />
          <Link href="/register" className="text-gray-700 hover:text-accent
hover:underline underline-offset-4 transition">
            Sign Up
          </Link>
        </>
      )}
    </div>
  );
}
```

Explanation:

The auth buttons component provides a comprehensive navigation and user management interface with proper dropdown styling and structure. It handles loading states with skeleton placeholders, provides conditional rendering based on authentication status, and includes admin dashboard access for privileged users. The dropdown menu uses custom styling classes and displays both user displayName and email in the header. Navigation includes "My Profile" at `/profile/user` route and proper logout functionality with router refresh. The unauthenticated state shows "Login" and "Sign Up" links with consistent styling.

Physical Code Explanation:

Line 1: `"use client"` directive for browser-side features. Lines 3-15: Import statements - React `useState` hook, authentication context `useAuth`, Next.js `Link` and `useRouter`, Radix UI dropdown components, and `ChevronDown` icon. Lines 17-20: Component function with router and auth context initialization, `dropdownOpen` state for controlling dropdown visibility. Lines 22-29: Loading state rendering - checks `auth.isLoading`, returns skeleton elements with `animate-pulse` class and gray background for visual feedback. Lines 31-50: Authenticated user navigation - "Sell Item" link to `/profile/new` with hover styles, visual separator `div` with gray line, "Cart" link to `/cart` with hover transitions. Lines 52-60: Dropdown menu structure - `DropdownMenu` with controlled `open` state, `DropdownMenuTrigger` with custom click handler, conditional styling based on `dropdownOpen` state using template literals and ternary operators, `ChevronDown` icon for visual dropdown indicator.

17 Custom Hooks

17.1 Session Timeout Hook (/src/hooks/useSessionTimeout.ts)

```typescript
const INACTIVITY_TIMEOUT = 15 * 60 * 1000 // 15 minutes in milliseconds
const ACTIVITY_KEY = 'lastActivity'
```

```
const STORAGE_EVENT_KEY = 'session-activity'

export function useSessionTimeout() {
  const router = useRouter()

  useEffect(() => {
    const updateActivity = () => {
      const now = Date.now().toString()
      localStorage.setItem(ACTIVITY_KEY, now)
      // Broadcast activity to other tabs
      localStorage.setItem(STORAGE_EVENT_KEY, now)
    }

    const checkSession = () => {
      const lastActivity = localStorage.getItem(ACTIVITY_KEY)

      if (lastActivity && auth.currentUser) {
        const timeSinceActivity = Date.now() - parseInt(lastActivity)

        if (timeSinceActivity > INACTIVITY_TIMEOUT) {
          // Auto logout after 15 minutes of inactivity
          localStorage.removeItem(ACTIVITY_KEY)
          localStorage.removeItem(STORAGE_EVENT_KEY)
          auth.signOut()
          router.push('/login?reason=session-expired')
        }
      }
    }

    const handleActivity = () => {
      if (auth.currentUser) {
        updateActivity()
      }
    }

    const handleStorageChange = (e: StorageEvent) => {
      // Sync activity across tabs
      if (e.key === STORAGE_EVENT_KEY && e.newValue) {
        localStorage.setItem(ACTIVITY_KEY, e.newValue)
      }
    }

    // Activity event listeners
    const activityEvents = [
      'mousedown',
      'mousemove',
      'keypress',
      'scroll',
      'touchstart',
      'click',
      'focus'
    ]
```

```
    // Set initial activity time when user is authenticated
    const setInitialActivity = () => {
      if (auth.currentUser) {
        updateActivity()
      }
    }

    // Initialize
    setInitialActivity()
    checkSession()

    // Add activity listeners
    activityEvents.forEach(event => {
      document.addEventListener(event, handleActivity, true)
    })

    // Listen for storage changes (cross-tab communication)
    window.addEventListener('storage', handleStorageChange)

    // Check session every minute
    const intervalId = setInterval(checkSession, 60 * 1000) // 1 minute

    // Listen for auth state changes
    const unsubscribe = auth.onAuthStateChanged((user) => {
      if (user) {
        updateActivity()
      } else {
        localStorage.removeItem(ACTIVITY_KEY)
        localStorage.removeItem(STORAGE_EVENT_KEY)
      }
    })

    return () => {
      clearInterval(intervalId)
      unsubscribe()

      // Remove activity listeners
      activityEvents.forEach(event => {
        document.removeEventListener(event, handleActivity, true)
      })

      window.removeEventListener('storage', handleStorageChange)
    }
  }, [router])
}
```

Explanation:

The session timeout hook implements comprehensive inactivity-based session management with cross-tab synchronization and Firebase auth state integration. It tracks user activity

through multiple event types including mouse, keyboard, touch, click, and focus events. The hook automatically sets initial activity on mount, manages localStorage cleanup on auth state changes, and provides robust cleanup in the useEffect return function. The implementation includes Firebase auth state listener integration, ensuring session management works seamlessly with the authentication system while maintaining consistent state across browser tabs.

Physical Code Explanation:

Lines 1-4: Import React `useEffect` hook, Next.js `useRouter` for navigation, and Firebase auth client. Lines 6-8: Constants definition - `INACTIVITY_TIMEOUT = 15 * 60 * 1000` sets 15-minute timeout in milliseconds, `ACTIVITY_KEY` and `STORAGE_EVENT_KEY` define localStorage keys for activity tracking and cross-tab communication. Lines 10-11: Hook function declaration and router initialization with `const router = useRouter()`. Lines 13-18: `updateActivity` function - gets current timestamp with `Date.now().toString()`, stores in localStorage with `localStorage.setItem(ACTIVITY_KEY, now)`, broadcasts to other tabs by setting storage event key. Lines 20-33: `checkSession` function - retrieves last activity with `localStorage.getItem(ACTIVITY_KEY)`, checks if user authenticated and activity exists, calculates time difference with `Date.now() - parseInt(lastActivity)`, compares against timeout threshold, performs logout if expired with `auth.signOut()` and navigation to login page. Lines 35-39: `handleActivity` function - checks user authentication with `auth.currentUser`, calls `updateActivity()` if user logged in. Lines 41-46: `handleStorageChange` function for cross-tab sync - listens for storage events, checks if event key matches `STORAGE_EVENT_KEY`, updates local activity timestamp from other tab's broadcast.

18 API Routes - Complete Implementation

18.1 Admin Status API (/src/app/api/admin/status/route.ts)

```typescript
export async function GET(request: NextRequest) {
  try {
    const authResult = await authenticateRequest(request)

    if (!authResult) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }

    // Return current admin status (this already checks environment + database)
    return NextResponse.json({
      isAdmin: authResult.admin || false,
      uid: authResult.uid,
      email: authResult.email
    })
  } catch (error) {
    console.error('Admin status check error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
```

```
}
```

Physical Code Explanation:

Line 1: `export async function GET(request: NextRequest) {` declares async GET handler for HTTP requests. Line 3: `const authResult = await authenticateRequest(request)` calls authentication helper function with request object, returns user data or null. Lines 5-7: Authentication validation - `if (!authResult)` checks if authentication failed, `return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })` returns JSON error response with 401 status code. Lines 9-15: Success response construction - comment explains admin status verification, `return NextResponse.json({` creates JSON response object, `isAdmin: authResult.admin || false` extracts admin claim with boolean fallback, `uid: authResult.uid` includes user ID, `email: authResult.email` includes user email address. Lines 16-19: Error handling - `catch (error)` block for exception handling, `console.error()` logs error details for debugging, returns 500 Internal Server Error with generic error message.

18.2 Cart Management API (/src/app/api/cart/route.ts)

```typescript
export async function GET(req: NextRequest) {
  try {
    const user = await authenticateRequest(req);
    if (!user) return NextResponse.json({ message: "Unauthorized" }, { status: 401 });

    const cartId = user.uid;
    const cartItemsSnap = await firestore.collection("cartItems")
      .where("cartId", "==", cartId)
      .get();

    const cartItems: CartItem[] = [];
    const itemIds: string[] = [];
    cartItemsSnap.forEach(doc => {
      const data = doc.data();
      cartItems.push({ cartItemId: doc.id, ...data } as CartItem);
      if (data.itemId) itemIds.push(data.itemId);
    });

    if (!itemIds.length) return NextResponse.json({ success: true, cartItems, itemsData: [] });

    const itemsSnap = await firestore.collection("items")
      .where(FieldPath.documentId(), "in", itemIds)
      .get();

    const itemsData: Item[] = [];
    itemsSnap.forEach(doc => {
      const data = doc.data();
      itemsData.push({
```

```
      id: doc.id,
      title: data.title,
      price: data.price,
      images: data.images || [],
      condition: data.condition || 'used',
      sellerId: data.sellerId || '',
      collectionAddress: data.collectionAddress || '',
      description: data.description || '',
      category: data.category || 'other',
      status: data.status || 'draft',
    } as Item);
  });

  // Filter out items that are not for sale and remove them from cart
  const validItems: Item[] = [];
  const validCartItems: CartItem[] = [];
  const itemsToRemove: string[] = [];

  for (let i = 0; i < itemsData.length; i++) {
    const item = itemsData[i];
    const cartItem = cartItems[i];

    if (item.status === 'for-sale') {
      validItems.push(item);
      validCartItems.push(cartItem);
    } else {
      itemsToRemove.push(item.id);
    }
  }

  // Remove invalid items from user's cart
  if (itemsToRemove.length > 0) {
    const batch = firestore.batch();

    for (const itemId of itemsToRemove) {
      const cartItemRef = firestore
        .collection('cartItems')
        .where('userId', '==', user.uid)
        .where('itemId', '==', itemId);

      const cartItemSnapshot = await cartItemRef.get();
      cartItemSnapshot.forEach(doc => {
        batch.delete(doc.ref);
      });
    }

    await batch.commit();
  }

  return NextResponse.json({
    success: true,
    cartItems: validCartItems,
```

```
      itemsData: validItems,
      removedItems: itemsToRemove.length
    });
  } catch (error) {
    console.error('Cart fetch error:', error);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500
});
  }
}
```

Explanation:

The cart API provides comprehensive cart management functionality with intelligent item validation and automatic cleanup. It retrieves cart items for authenticated users and performs batch fetching of item details using Firestore's `in` query operator. The implementation includes sophisticated item status validation, automatically removing items that are no longer "for-sale" from user carts to maintain data integrity. The response includes cart metadata, validated item information, and cleanup statistics, ensuring users only see purchasable items in their carts.

Physical Code Explanation:

Lines 1-3: Import Next.js request/response objects, Firebase server utilities (`firestore`, `authenticateRequest`), and Firestore `FieldPath` for document ID queries. Lines 5-11: `CartItem` interface defining cart item structure with `cartItemId`, `itemId`, and `cartId` properties. Lines 13-23: `Item` interface defining item data structure with required fields (`id`, `title`, `price`, `images`) and optional properties (`condition`, `sellerId`, etc.) using TypeScript optional property syntax with `?`. Lines 25-27: `GET` function declaration for handling cart retrieval requests, authentication check with `authenticateRequest(req)`, return 401 Unauthorized if no valid user. Lines 29-37: Cart items retrieval - use user UID as cart ID with `const cartId = user.uid`, query Firestore collection `cartItems` with `where("cartId", "==", cartId)`, iterate through results with `forEach()` to build `cartItems` array and extract `itemIds` for batch fetching. Lines 39-47: Batch item details fetching - early return with empty data if no items, use Firestore `in` query with `FieldPath.documentId()` to fetch multiple items efficiently, iterate through item documents to build `itemsData` array with proper type mapping and default values for optional fields.

19 Utility Functions

19.1 Image URL Formatter (/src/lib/imageUrlFormatter.ts)

```typescript
export default function imageUrlFormatter(imagePath: string){
    return
`https://firebasestorage.googleapis.com/v0/b/${process.env.NEXT_PUBLIC_FIREBAS
E_STORAGE_BUCKET}/o/${encodeURIComponent(imagePath)}?alt=media`;
}
```

Explanation:

The image URL formatter creates properly encoded Firebase Storage URLs for image access. It uses the configured storage bucket from environment variables and properly encodes the image path to handle special characters and spaces in filenames. The `alt=media` parameter ensures direct image serving without Firebase Storage metadata.

Physical Code Explanation:

Line 1: `export default function imageUrlFormatter(imagePath: string){` declares function with TypeScript parameter type annotation. Line 2: `return \`https://firebasestorage.googleapis.com/v0/b/${process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET}/o/${encodeURIComponent(imagePath)}?alt=media\`;` constructs Firebase Storage URL using template literal with `${}` interpolation - `process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET` accesses environment variable, `encodeURIComponent(imagePath)` properly encodes the file path to handle special characters and spaces, `?alt=media` query parameter tells Firebase to serve raw file content instead of metadata JSON. Line 3: `}` closes the function.

20 Firebase Storage Security Rules (/storage.rules)

20.1 Production Storage Rules Implementation

```javascript
// Firebase Storage Security Rules
// These rules should be deployed to Firebase Storage

rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {

    // Allow authenticated users to read all images
    match /{allPaths=**} {
      allow read: if request.auth != null;
    }

    // Item photos - users can upload to their own items
    match /items/{itemId}/{fileName} {
      allow write: if request.auth != null
        && isValidImage()
        && isOwnerOfItem(itemId);
      allow delete: if request.auth != null
        && isOwnerOfItem(itemId);
    }

    // Profile pictures - users can upload their own
    match /profiles/{userId}/{fileName} {
      allow write: if request.auth != null
        && request.auth.uid == userId
        && isValidImage();
      allow delete: if request.auth != null
        && request.auth.uid == userId;
```

```
    }

    // Helper functions
    function isValidImage() {
      return request.resource.contentType.matches('image/.*')
        && request.resource.size < 5 * 1024 * 1024; // 5MB limit
    }

    function isOwnerOfItem(itemId) {
      // This would need to be verified against Firestore
      // For now, allow any authenticated user
      // In production, you'd want to check the item ownership
      return request.auth != null;
    }
  }
}
```

Explanation:

These Firebase Storage security rules implement a comprehensive access control system for the marketplace's file uploads. The rules enforce authentication for all read operations, ensuring only logged-in users can access uploaded images. For item photos, the rules restrict write access to authenticated users with valid images under 5MB, while delete permissions require item ownership verification. Profile pictures follow a strict user-ownership model where users can only upload/delete their own profile images. The `isValidImage()` helper function validates file types and sizes, preventing abuse and ensuring optimal performance. The `isOwnerOfItem()` function provides a foundation for item ownership verification that should be enhanced with Firestore queries in production. These rules protect against unauthorized uploads, malicious file types, and oversized files while maintaining usability for legitimate users.

Physical Code Explanation:

Lines 1-3: Comments explaining purpose and deployment context. Line 5: `rules_version = '2';` specifies Firebase Security Rules version 2 syntax. Line 6: `service firebase.storage {` begins storage service rule definition. Line 7: `match /b/{bucket}/o {` matches all objects in any storage bucket with wildcard `{bucket}`. Lines 9-11: Global read rule - `match /{allPaths=**}` uses recursive wildcard `**` to match all paths at any depth, `allow read: if request.auth != null;` permits read access only for authenticated users. Lines 13-19: Item photos rules - `match /items/{itemId}/{fileName}` captures itemId and fileName variables, `allow write: if request.auth != null && isValidImage() && isOwnerOfItem(itemId);` requires authentication, valid image format, and ownership verification, `allow delete` with similar conditions. Lines 21-27: Profile pictures rules - `match /profiles/{userId}/{fileName}` captures userId, `request.auth.uid == userId` ensures users can only access their own profile images. Lines 29-35: `isValidImage()` helper function - `request.resource.contentType.matches('image/.*')` validates MIME type using regex, `request.resource.size < 5 * 1024 * 1024` enforces 5MB size limit. Lines 37-42: `isOwnerOfItem()` helper function with TODO comment for Firestore integration, currently allows any authenticated user.

```
    // Helper functions
    function isValidImage() {
      return request.resource.contentType.matches('image/.*')
        && request.resource.size < 5 * 1024 * 1024; // 5MB limit
    }

    function isOwnerOfItem(itemId) {
      // This would need to be verified against Firestore
      // For now, allow any authenticated user
      // In production, you'd want to check the item ownership
      return request.auth != null;
    }
  }
}
```

21 Environment Variables Configuration (/.env.example)

21.1 Complete Environment Template

```bash
# Firebase Environment Variables
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
NEXT_PUBLIC_FIREBASE_MEASUREMENT_ID=
#NEXT_PUBLIC_BASE_URL=http://localhost:3000  # Local development URL

# Firebase Admin SDK (Server-side only - NOT prefixed with NEXT_PUBLIC_)
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
FIREBASE_PRIVATE_KEY=

# PayFast Payment Gateway (Add these to your .env.local)
# PAYFAST_MERCHANT_ID=your_merchant_id
# PAYFAST_MERCHANT_KEY=your_merchant_key
# PAYFAST_PASSPHRASE=your_passphrase
```

```
# Admin Configuration
# ADMIN_EMAILS=admin1@example.com,admin2@example.com

# Application Settings
# NODE_ENV=development
```

Explanation:

This environment configuration template provides developers with all necessary environment variables for local development setup. The Firebase client variables prefixed with `NEXT_PUBLIC_` are exposed to the browser for direct Firebase SDK communication. The Firebase Admin SDK variables remain server-side only for security. PayFast and admin configuration are commented out as examples, requiring developers to uncomment and configure with their actual credentials. This separation ensures sensitive production credentials never accidentally get committed to version control while providing clear guidance for required configuration.

Physical Code Explanation:

Lines 1-8: Firebase client configuration section with `# Firebase Environment Variables` comment header, variables prefixed with `NEXT_PUBLIC_` for browser exposure - `NEXT_PUBLIC_FIREBASE_API_KEY=` for Firebase project API key, `NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=` for authentication domain, similar pattern for project ID, storage bucket, messaging sender ID, app ID, and measurement ID, all followed by `=` with empty values for user completion. Line 9: `#NEXT_PUBLIC_BASE_URL=http://localhost:3000` commented out with `#` prefix, showing local development URL example. Lines 11-14: Firebase Admin SDK section with comment header, server-side only variables without `NEXT_PUBLIC_` prefix for security - `FIREBASE_PROJECT_ID=`, `FIREBASE_CLIENT_EMAIL=`, `FIREBASE_PRIVATE_KEY=` all empty for user configuration. Lines 16-19: PayFast section with comment explaining local file requirement, variables commented out with `#` prefix - `# PAYFAST_MERCHANT_ID=your_merchant_id` shows example format. Lines 21-22: Admin configuration with `# ADMIN_EMAILS=admin1@example.com,admin2@example.com` showing comma-separated email format. Lines 24-25: Application settings with `# NODE_ENV=development` for environment specification.

22 Deployment Instructions (/firebase.json)

22.1 Firebase Hosting Configuration

```json
{
  "firestore": {
    "rules": "firestore.rules",
    "indexes": "firestore.indexes.json"
  },
  "storage": {
    "rules": "storage.rules"
  },
```

```
  "hosting": {
    "source": ".",
    "ignore": [
      "firebase.json",
      "**/.*",
      "**/node_modules/**"
    ],
    "frameworksBackend": {
      "region": "europe-west1"
    }
  }
}
```

22.2 Production Deployment Steps

```bash
# 1. Install Firebase CLI globally
npm install -g firebase-tools

# 2. Login to Firebase
firebase login

# 3. Initialize project (if not already done)
firebase init

# 4. Build the Next.js application
npm run build

# 5. Deploy to Firebase Hosting
firebase deploy

# 6. Deploy only specific services (optional)
firebase deploy --only hosting
firebase deploy --only firestore:rules,storage:rules
```

Explanation:

This deployment workflow ensures reliable production releases with proper validation and rollback capabilities. The Firebase CLI installation provides comprehensive tooling for project management, deployment automation, and monitoring. The authentication step establishes secure credentials for deployment operations. The build process compiles and optimizes the Next.js application, generating production-ready assets with code splitting, image optimization, and performance enhancements. Full deployment updates all services simultaneously, ensuring consistency across hosting, database rules, and storage policies. Selective deployment options enable targeted updates for specific services, crucial for production environments where you might need to update security rules without triggering application redeployment. This approach supports continuous integration workflows, automated testing, and gradual rollout strategies essential for marketplace platforms serving

real users and processing actual payments. The Firebase hosting configuration uses the frameworksBackend feature optimized for Next.js applications, with the backend region set to europe-west1 for optimal performance. The ignore patterns prevent deployment of development files and dependencies. The deployment process leverages Firebase's automatic Next.js integration, handling server-side rendering, API routes, and static asset optimization. The separate deployment commands allow granular control over which services get updated, essential for production environments where you might want to update rules without touching the application code.

Physical Code Explanation:

Line 2: `npm install -g firebase-tools` installs Firebase CLI globally with `-g` flag for system-wide access. Line 5: `firebase login` opens browser authentication flow to authenticate with Google account. Line 8: `firebase init` initializes Firebase project configuration, creates `firebase.json` and other config files. Line 11: `npm run build` executes build script from `package.json`, compiles Next.js application for production. Line 14: `firebase deploy` deploys all configured services (hosting, functions, rules) to Firebase. Lines 17-18: `firebase deploy --only hosting` deploys specific service using `--only` flag, `--only firestore:rules,storage:rules` deploys multiple specific services separated by commas.

23 API Error Handling Patterns

23.1 Standardized Error Response Format

```typescript
// Consistent error responses across your API routes

// Authentication Errors (401)
return NextResponse.json({ message: "Unauthorized" }, { status: 401 });

// Permission Errors (403)
return NextResponse.json({ message: "Forbidden" }, { status: 403 });

// Validation Errors (400)
return NextResponse.json({ message: "Invalid item IDs" }, { status: 400 });
return NextResponse.json({ message: "Missing user id" }, { status: 400 });

// Not Found Errors (404)
return NextResponse.json({ message: "User not found in Firestore" }, { status:
404 });

// Server Errors (500)
return NextResponse.json({
  success: false,
  message: errorMessage
}, { status: 500 });
```

23.2 Error Handling Implementation Examples

```typescript
// From /src/app/api/users/[id]/route.ts
try {
  // API logic here
} catch (error) {
  const errorMessage = error instanceof Error ? error.message : 'Unknown error';
  return NextResponse.json({
    success: false,
    message: errorMessage
  }, { status: 500 });
}

// From /src/app/api/users/suspend/route.ts - Admin validation
if (!adminUser) return NextResponse.json({ message: "Unauthorized" }, { status: 401 });
if (!adminUser.admin) return NextResponse.json({ message: "Forbidden" }, { status: 403 });

// From /src/app/api/payments/process-success/route.ts - Payment validation
if (payment_status !== 'COMPLETE') {
  return NextResponse.json({ message: "Payment not successful" }, { status: 400 });
}
```

Explanation:

Your API routes implement consistent error handling patterns with appropriate HTTP status codes and descriptive error messages. The authentication flow uses 401 for missing auth and 403 for insufficient permissions. Validation errors return 400 with specific messages about missing or invalid data. Server errors are caught and return 500 status with error details. This standardization enables frontend components to handle errors predictably and provide appropriate user feedback. The error messages are descriptive enough for debugging while remaining user-friendly. The try-catch patterns ensure graceful degradation when external services fail, while the admin validation demonstrates proper authorization checking. Payment validation ensures transaction integrity by verifying completion status before processing orders. This comprehensive error handling strategy supports reliable marketplace operations and provides clear debugging information for development and production monitoring.

Physical Code Explanation:

Lines 3-11: Try-catch error handling pattern - `catch (error)` block extracts error message using `error instanceof Error ? error.message : 'Unknown error'` for type-safe error handling, constructs response object with `success: false` and `message: errorMessage`, returns 500 Internal Server Error status. Lines 14-15: Admin validation examples use early returns with appropriate status codes - `if (!adminUser)` checks user existence, `if (!adminUser.admin)` verifies admin permissions, both return specific HTTP status codes with descriptive messages. Lines 17-19: Payment validation checks specific conditions like `payment_status

!== 'COMPLETE'` using strict comparison operator `!==`, returns 400 Bad Request with descriptive error message for failed payments.

This comprehensive documentation now accurately covers the real implementations that exist in your Campus Marketplace platform, providing developers with complete guidance for understanding, maintaining, and extending the actual application components and functionality.